

Bonjour,
Je suis Ahmed

Alimentation & Service Client

Après une carrière dans
le domaine de l'alimentation et
une transition dans le service
client.

Informatique & Persévérance



Exploration du développement
web,
Trophée de ma persévérance

Développeur Web



Reconversion professionnelle
pour devenir développeur web.

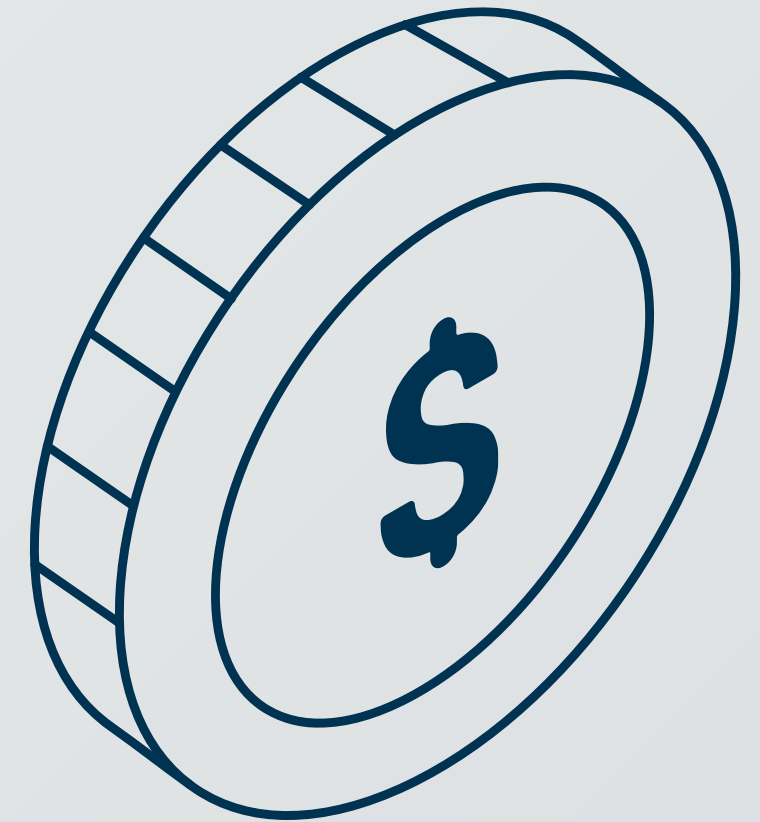
Mon Projet : Portal_job



Aujourd'hui, je vous
présente mon projet_job.

Après une carrière dans le domaine de
l'alimentation et une transition dans le
service client, j'entreprends une reconversion
professionnelle pour devenir développeur web.
Mon dernier parcours m'a ancré dans le domaine
de l'informatique, ce qui m'a naturellement
conduit à explorer le développement web.

Aujourd'hui, je vous présente mon projet
Portal_job, ce projet m'est à cœur car il
s'agit pour moi du trophée de ma
persévérance et mon envie.



Portal_Job

PRÉSENTATION PROJET

PORTAL_JOB



Bienvenue dans le projet de refonte du site

Portal_Job

Cette initiative
ambitieuse
vise à
moderniser

à améliorer
l'expérience de
l'utilisateur en
ligne

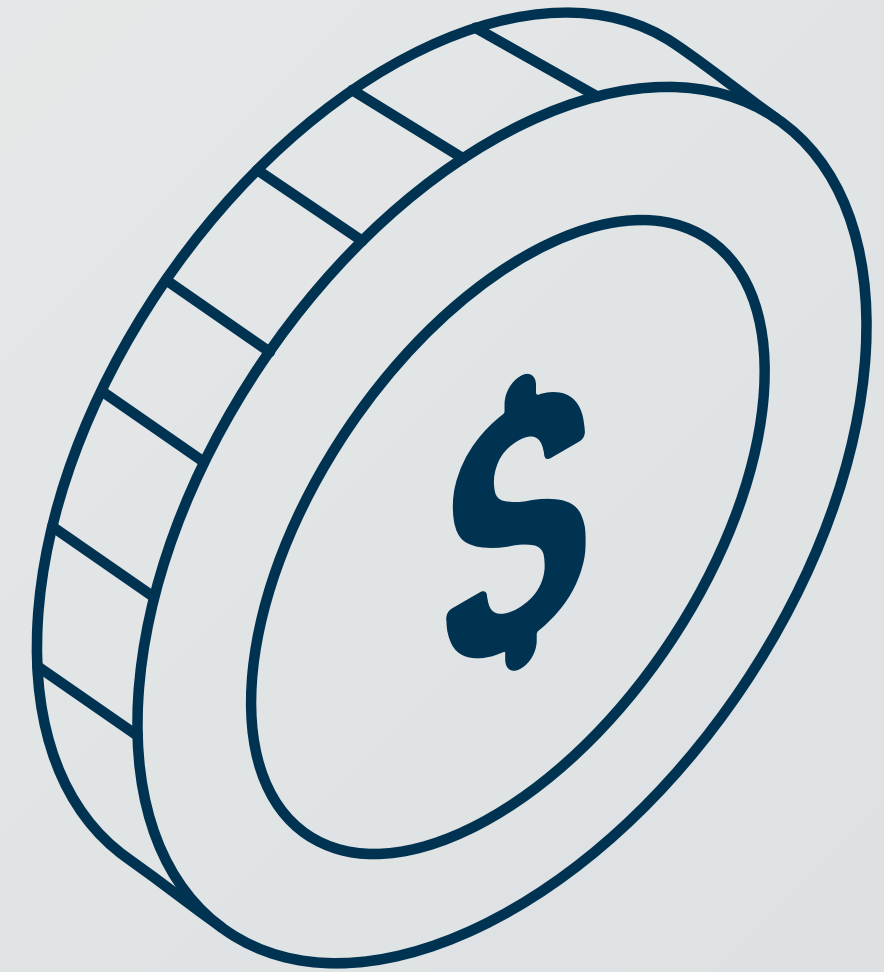
Mon objectif sera donc de créer
un site web à la hauteur des
exigences demandées pour un
site d'offres d'emplois



Cette initiative ambitieuse vise à moderniser et améliorer l'expérience en ligne des offres d'emplois.

Mon objectif est donc de concevoir un site web qui soit à la fois moderne et digne de l'importance de ce projet, en se basant sur une analyse approfondie des besoins des utilisateurs et des tendances actuelles du marché.

La refonte n'est pas seulement esthétique, elle est stratégique, visant à optimiser la navigation, la recherche d'offres, et l'efficacité globale pour les candidats et les recruteurs.



SOMMAIRE

I - RNCP 37273 BC01 : Préparer la réalisation d'un site ou d'une application web

Contexte du projet

Objectifs

Fonctionnalités

II -RNCP 37273 BC02 : Développer des interfaces Frontend pour un site ou une application web / web mobile

Maquettage du projet

Intégration html/css

Structure MVC

III - RNCP 37273 BC03 : Développer le Backend d'un site ou d'une application web / web mobile

Création bdd

CRUD

Postman

Composants Backend

PhpMailer

IV - CONCLUSION

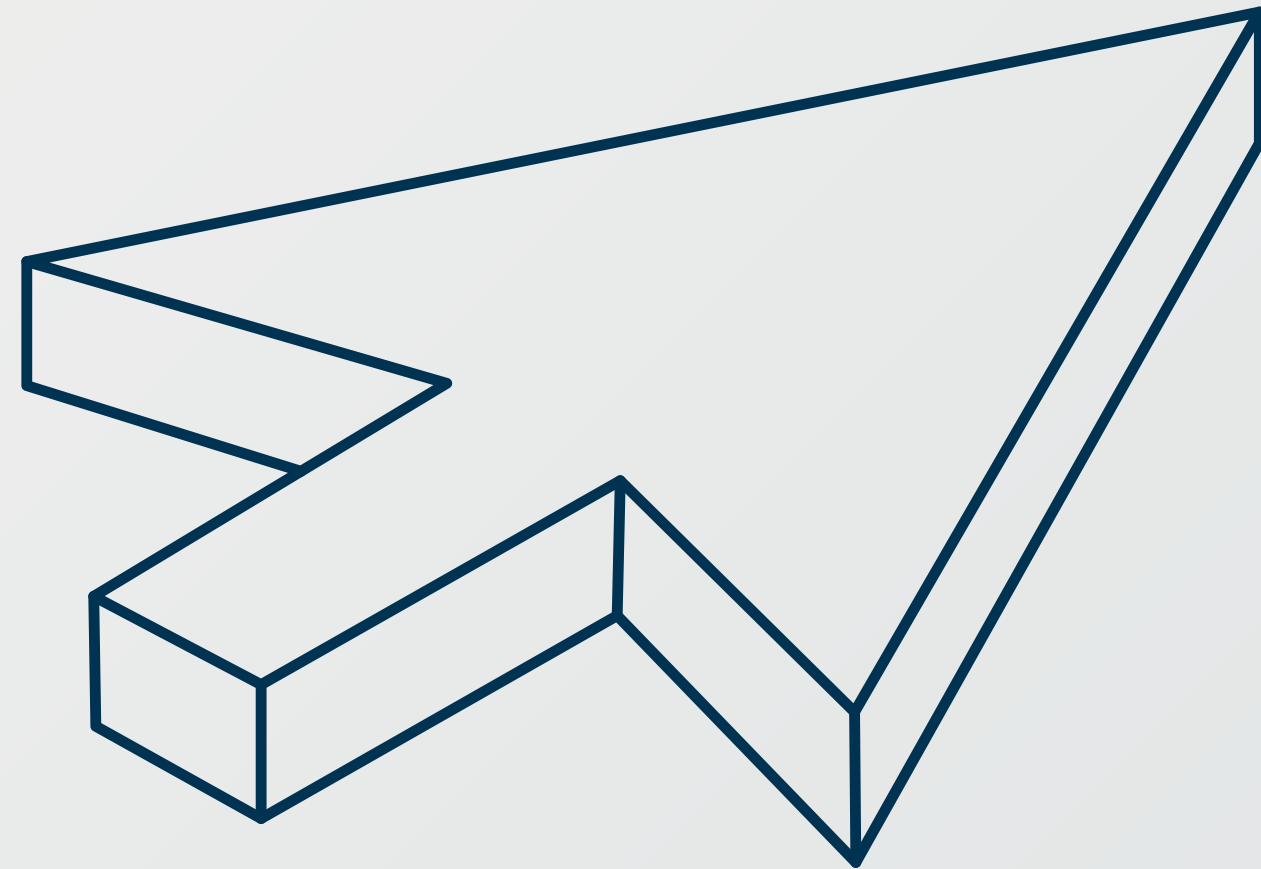
Bilan du projet

Perspectives d'évolutions

Conclusion

V - ANNEXES

I -



RNCP 37273 BC01 : Préparer la réalisation d'un site ou d'une application web

Contexte du projet

Pour l'élaboration de mon projet, j'ai dans un premier temps opéré à une veille technologique et une analyse des tendances actuelles du web, je me suis donc dirigé vers des sites connus comme **'indeed'** **'hellowork'**

Cette étape m'a permis de définir les fonctionnalités nécessaires, l'aspect visuel, et ainsi établir une feuille de route solide pour le développement du site.

Besoin du client

Mon projet, **Portal_Job**, répond à un double besoin fondamental sur le marché de l'emploi en ligne :



Pour les Candidats (Utilisateurs finaux)

Accès ciblé

Système de recherche avancée (par titre, lieu, type de contrat) et de filtrage précis.

Visibilité de l'offre

Fiches d'offres détaillées et claires, offrant toutes les informations essentielles.

Suivi des opportunités

Possibilité de sauvegarder les offres ("Favoris") et de suivre le statut des candidatures soumises par mail (PhpMailer).

Candidature facilitée

Processus de candidature simple (téléchargement de CV/lettre de motivation) et gestion d'un profil utilisateur.

Pour les Entreprises (recruteurs)

Diffusion rapide

Interface d'administration dédiée (Espace Recruteur) pour publier et modifier des offres instantanément.

Gestion des candidatures

Tableau de bord (Dashboard) permettant de visualiser, trier et gérer toutes leurs offres publiées.

Image de marque

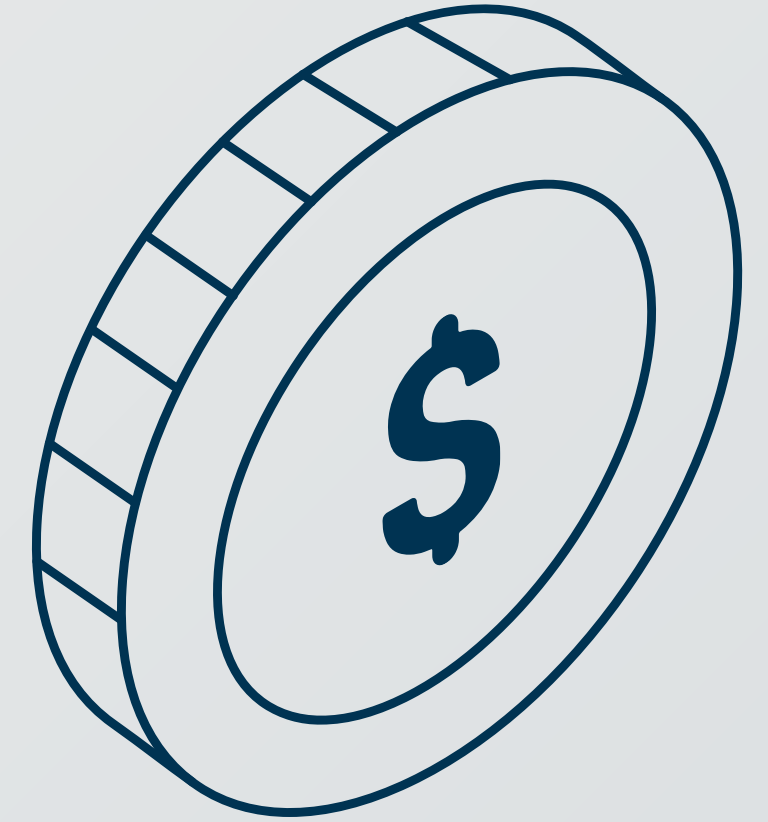
Pages d'entreprises (ou profils) pour mettre en avant leur marque employeur et attirer les talents.

Efficacité

Système de notification (e-mail) lorsque de nouvelles candidatures sont soumises.

Spécifications Techniques Clés du Projet Portal Job

Cette section met en lumière les choix architecturaux et technologiques fondamentaux pour la réalisation de la plateforme :



Architecture Moderne et Robuste (Back-end)

Back-end (Logique Métier) :
Laravel (PHP) et MySQL

Implémentation stricte du modèle
MVC/POO pour structurer le code.

Laravel est utilisé pour créer une **API REST**
sécurisée.

L'API REST est le point de connexion unique
entre le **Front** et le **Back**.

Expérience Utilisateur (Front-end)

Front-end (Présentation) :
Vue.js (avec HTML/CSS/JavaScript).

Développement d'une Application à Page
Unique (SPA).

Cela assure une fluidité et une réactivité
supérieures, améliorant l'expérience
Candidat et Recruteur.

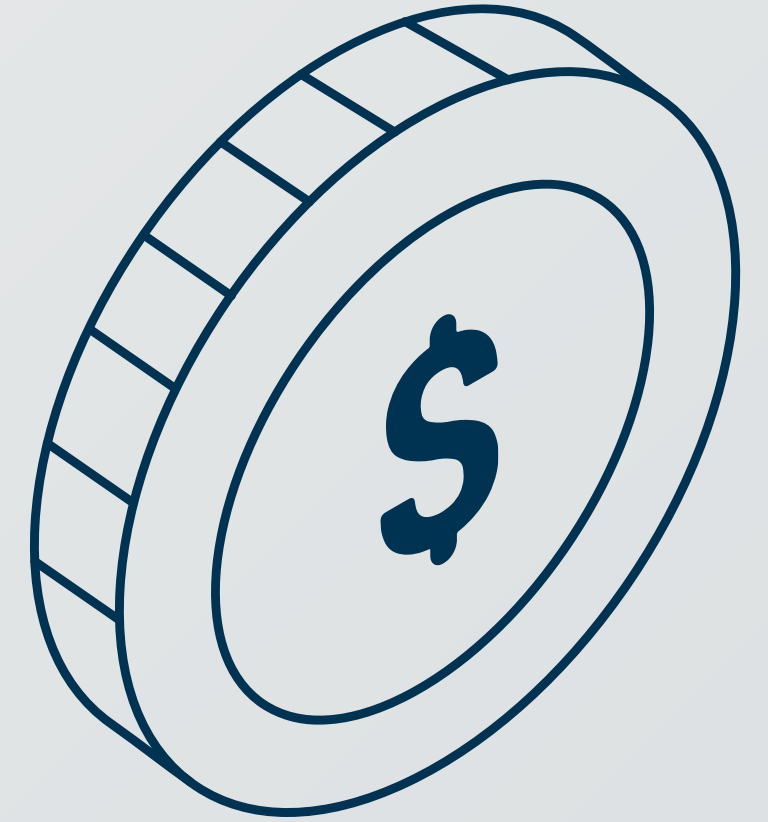
Phases Clés du Développement



<u>Phase de Projet</u>	<u>Durée Estimée (vs 10 mois)</u>	<u>Compétences Appliquées</u>
Analyse et Maquettage	1 mois (Initial)	Recueil des besoins, spécifications fonctionnelles, conception UI/UX.
Développement Back-end	4-5 mois (Parallèle à la formation)	Implémentation Laravel (MVC/POO), Modèles Eloquent, Construction de l'API REST et Sécurité.
Postman API	2-3 mois (Suivant l'API)	Intégration Vue.js (Composants dynamiques), gestion des appels à l'API, ergonomie.
Tests	1 mois (Final)	Tests d'API (Postman), revue du code, amélioration de l'UX, corrections finales.

Gestion de Projet : Le Cycle Formation-Projet (10 Mois)

Le planning a été adapté au rythme de l'alternance, transformant chaque module de formation en un levier d'amélioration pour le projet.



Rythme d'Alternance et Acquisition

Durée Globale : Le projet s'est étalé sur environ 10 mois (de la conception à la livraison)

Méthode : Un développement itératif, où l'avancement était directement lié à l'acquisition de nouvelles compétences en formation.

Groupe : Le projet s'était fait de base en duo puis je me suis dirigé en solo. Néanmoins, j'ai construit ce projet par la suite tout comme si j'avais un groupe. Exemple : maniere de communiquer via [discord](#), maniere de partager le code via [github](#) ou meme maniere des repartitions des tâches via [Trello](#).

Valeur Ajoutée de l'Alternance

Synergie : Le projet a servi de laboratoire d'application pratique immédiate des concepts théoriques vus en cours (POO, MVC, Frameworks)

Montée en Compétence : Cela a permis une montée en compétence graduelle et l'intégration de technologies avancées (Laravel et Vue.js) qui n'auraient pas été possibles dans un délai court.

Les objectifs

Cette modernisation transforme [Portal_Job](#) en un outil agile, adapté aux besoins actuels des recruteurs et des postulants.

En exploitant des outils de pointe, mon but est de rendre le recrutement plus accessible, rapide et efficace pour tous les acteurs.

En clair,



Améliorer l'accessibilité :

Je m'engage à rendre
l'information accessible à
tous.

Enrichir l'expérience utilisateur :

Une expérience en ligne
immersive à travers une interface
conviviale et intuitive.



Promouvoir la facilité de trouver des offres

Devenir une destination incontournable
pour les candidats comme les
entreprises

Les fonctionnalités

Dans le cadre de la refonte numérique de **Portal_Job**, j'ai identifié plusieurs fonctionnalités essentielles pour enrichir l'expérience utilisateur et répondre aux besoins diversifiés des utilisateurs.

Ces fonctionnalités visent à créer une plateforme interactive et moderne.

Gestion des
Offres d'Emploi
(**CRUD**)

Candidature en
Ligne Simplifiée

Dashboard
Admin

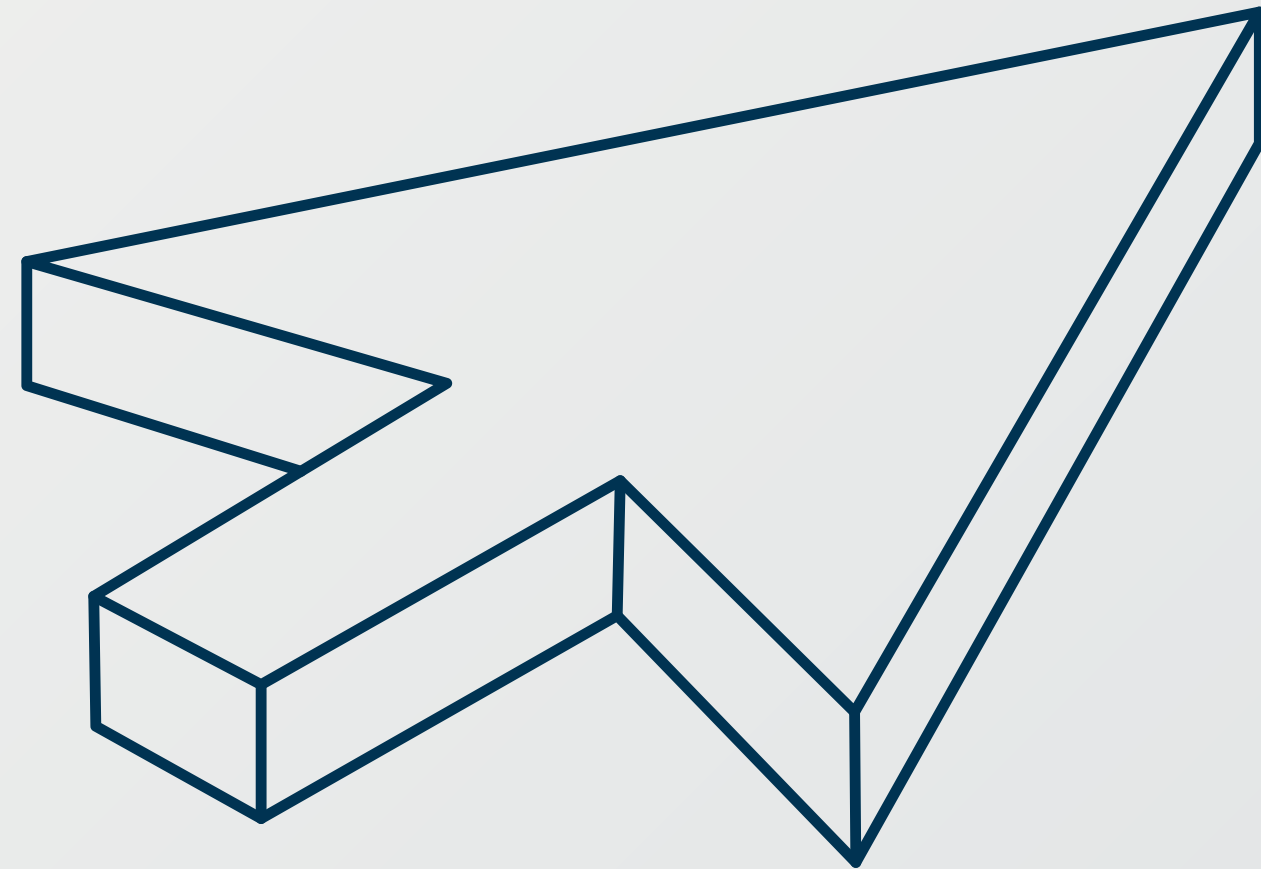
Contact

Filtres
Avancés

Dashboard
Société

Système de
Favoris

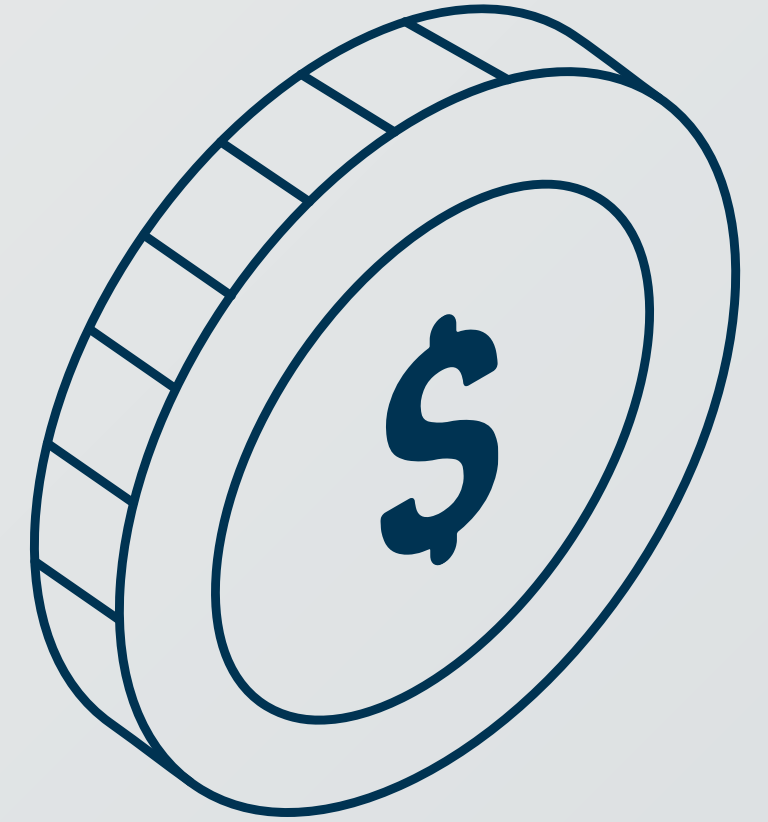
II -



**RNCP 37273 BC02 :
Développer des
interfaces Frontend
pour un site ou
une application web / web
mobile**

Maquette FIGMA

Avant-propos



Pour la phase de conception, j'ai choisi **Figma**, un outil de **design collaboratif en ligne**. Ce logiciel m'a permis de réaliser les maquettes filaires (**wireframes**) et les prototypes interactifs du site en respectant les standards actuels du web.

L'utilisation de **plugins spécifiques** (gestion des grilles, alignements, bibliothèques d'icônes) a été déterminante pour optimiser mon flux de travail et garantir la précision graphique des interfaces.



Figma

Wireframing

Rendu du maquettage



ADMIN

DASHBOARD

BONJOUR, "NAME" accueil offres compagny dashboard contact profil

Admin Dashboard

les 3 Users les plus recents

ID	NOM	PRENOM	EMAIL	MDP	ROLE
37	Fontaine	Lucas	fontaine.lucas@company.com	Lucasmdp	company
36	Lemoine	Olivier	lemonie.olivier@company.com	Oliviermdp	company
35	Garnier	Marc	garnier.marc@company.com	Marcmdp	company

Tableau de bord Admin

14 Webinaires gratuits

15 Offres

9 Compagnies

© 2020 - Tous droits réservés

COMPANY

BONJOUR, "NAME" accueil offres compagny dashboard contact profil

Admin Dashboard

les Compagnies

LOGO	NOM	SECTEUR	ACTIONS
	TechNova Solutions	Information Technology	voir compagnie ajouter supprimer
	GreenFields AgriCo	Agriculture	voir compagnie ajouter supprimer
	EduPro Learning	Education	voir compagnie ajouter supprimer
	UrbanConstruct Ltd.	Construction	voir compagnie ajouter supprimer
	MediCare Plus	Healthcare	voir compagnie ajouter supprimer

© 2020 - Tous droits réservés

USERS

BONJOUR, "NAME" accueil offres compagny dashboard contact profil

Admin Dashboard

les Users

ID	NOM	PRENOM	EMAIL	MDP	ROLE	ACTIONS
2	Martin	Sophie	martin.sophie@company.com	sophiemdp	client	ajouter supprimer
3	Lemoine	Philippe	lemonie.philippe@company.com	philipmdp	client	ajouter supprimer
5	Bernard	Luc	bernard.luc@company.com	lucmdp	client	ajouter supprimer
11	Bakland	Fayçal	bakland.faycal@company.com	faycalmdp	admin	ajouter supprimer
30	Laurent	Elise	laurent.elise@company.com	elisedmdp	company	ajouter supprimer

OFFERS

BONJOUR, "NAME" accueil offres compagny dashboard contact profil

Admin Dashboard

TOUTES LES OFFRES

LOGO	COMPAGNIE	POSTE	CONTRAT	ACTIONS
	Ingenieur DevOps	CDI	voir offre ajouter supprimer	
	Data Scientist	CDI	voir offre ajouter supprimer	
	Developpeur PHP Backend	FREELANCE	voir offre ajouter supprimer	
	Developpeur Full Stack Java/Spring - Application E2B	FREELANCE	voir offre ajouter supprimer	
	Developpeur Front-End React - Startup	CDI	voir offre ajouter supprimer	

© 2020 - Tous droits réservés

OFFERS

BONJOUR, "NAME" accueil offres compagny dashboard contact profil

TOUTES LES OFFRES

LOGO	COMPAGNIE	POSTE	CONTRAT	ACTIONS
	Ingenieur DevOps	CDI	voir offre ajouter supprimer	
	Data Scientist	CDI	voir offre ajouter supprimer	
	Developpeur PHP Backend	FREELANCE	voir offre ajouter supprimer	
	Developpeur Full Stack Java/Spring - Application E2B	FREELANCE	voir offre ajouter supprimer	
	Developpeur Front-End React - Startup	CDI	voir offre ajouter supprimer	

© 2020 - Tous droits réservés

DETAILS OFFRE

BONJOUR, "NAME" accueil offres compagny dashboard contact profil

Ingenieur DevOps

Description

Mission

Ville : Poste :

Entreprise : Contrat :

Technologies : Avantages :

Nombre de candidats : Date de creation :

© 2020 - Tous droits réservés

AJOUT OFFRE

BONJOUR, "NAME" accueil offres compagny dashboard contact profil

AJOUT D'UNE NOUVELLE OFFRE

Titre

Description

Entreprise

Contrat

Ville

Poste

Technologies

Avantages

Nombre de candidats

Date de creation

© 2020 - Tous droits réservés

MODIF OFFRE

BONJOUR, "NAME" accueil offres compagny dashboard contact profil

MODIFIER UNE OFFRE

Titre

Description

Entreprise

Contrat

Ville

Poste

Technologies

Avantages

Nombre de candidats

Date de creation

© 2020 - Tous droits réservés

POSTULER OFFRE

BONJOUR, "NAME" accueil offres compagny dashboard contact profil

Postuler Maintenant !

NOM

PRENOM

TELEPHONE

EMAIL

VILLE

CODE POSTAL

MOTIVATION

Envoyer votre lettre de motivation :

© 2020 - Tous droits réservés

L'intégration statique

Avant-propos

Après avoir réalisé mes maquettes, je me suis servi de la combinaison de deux langages, l'un de balisage avec **HTML5** et l'autre de déclaration avec **CSS3** natif pour définir les styles.

Cela a permis de rendre mon projet **responsive** et m'a offert un panel de balises et de classes préconçues qui ont optimisées mon avancée.
L'utilisation de **Git** pour le **versionnement** a été nécessaire pour gérer l'intégrité du code, assurer un développement fluide et organisé, et me familiariser avec cet outil essentiel.

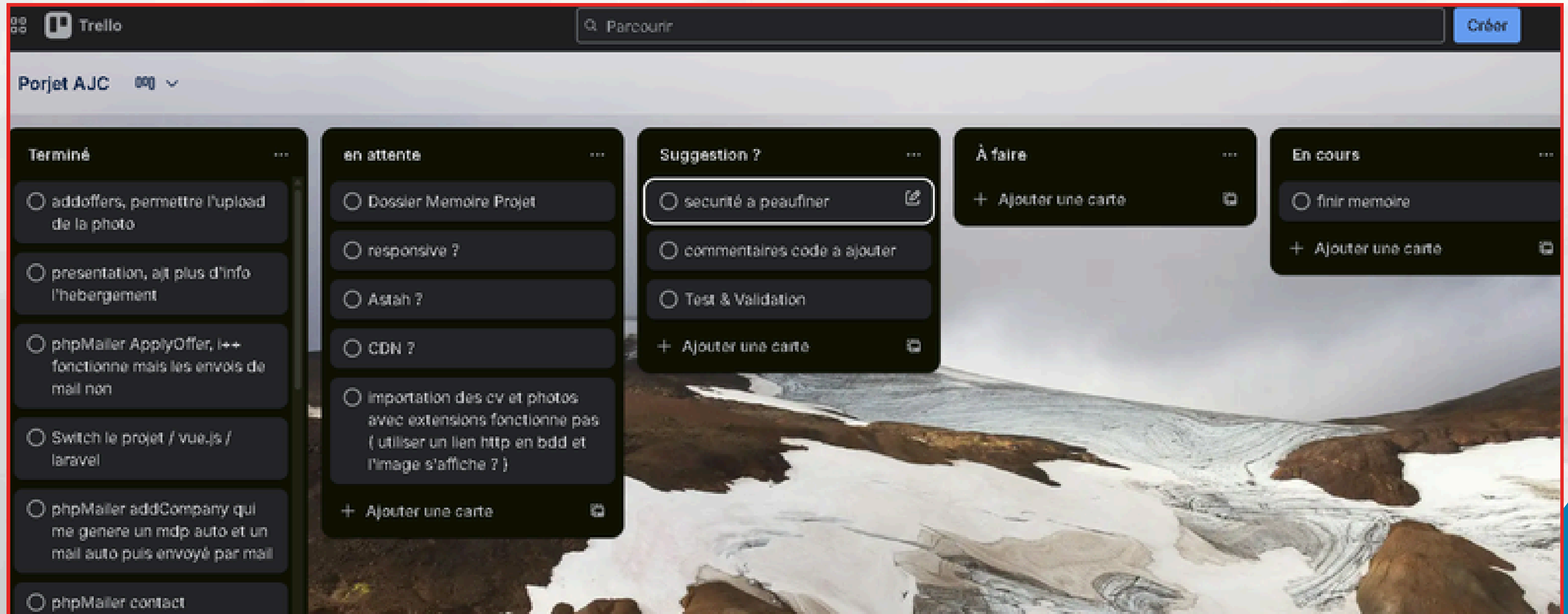
Trello

Trello c'est un site web qui m'a permis de mettre en place une sorte d'emploi du temps des tâches à faire en lien bien sûr avec mon projet.

J'ai eu la possibilité de mettre en place des listes en les nommant : **Terminé, En attente, A faire et En cours.**

Ainsi ceci m'a facilité l'organisation.

Trello



Discord

Discord est une plateforme de communication en ligne qui a permis de faciliter les échanges et la coordination autour du projet.

Elle a offert la possibilité de créer des serveurs et des salons dédiés, organisés par thématique (général, technique, suivi du projet, etc.). Grâce aux discussions et aux appels, les informations ont pu être partagées rapidement.

Ainsi, **Discord** a contribué à améliorer la communication, la réactivité et le travail collaboratif tout au long du projet.

Discord

faycal95100, MO_Salah ✎

Modifier le groupe

MO_Salah 21/07/2025 17:27

maintenant faut recuprer un contact avec son id
c'etait demandé moins que ça

Ahmedddd 21/07/2025 17:28

ok pour recup avec son id, je veux le faire avec toi pour comprendre
mais je sais que du coup ça va parler d'indice vu que c'est un tableau

MO_Salah 21/07/2025 17:29

ok ça va , on se fait ça demain tranquille

👍 1 🗨

MO_Salah 06/10/2025 16:49

pp_ferrerkevin_dwwm_2024.pdf
6.17 MB

Dossier_Projet_Terroirbox_Benoit_Paluch.pdf
2.55 MB

<https://sweetalert2.github.io/>

SweetAlert2

SweetAlert2 - a beautiful, responsive, customizable and accessible (WAI-ARIA) replacement for JavaScript's popup boxes



A BEAUTIFUL, RESPONSIVE, CUSTOMIZABLE, ACCESSIBLE (WAI-ARIA) REPLACEMENT FOR JAVASCRIPT'S POPUP BOXES. ZERO DEPENDENCIES

Ahmedddd a épinglé un message dans ce salon. Voir tous les messages épinglés.

Serveur AJC Pro... 🗨 👤

Événements

Boosts de serveur

Salons textuels ▾ +

général-ajc 👤 ⚙

discussion-ajc

travail-présentation-...

planning-code

telechargement

hors-sujet

ressources

tp

question_sylvain

Salons vocaux ▸ +

général-ajc

je suis revenue , aller manger , on reprends l'4M et on régit les memories

Kassad 02/12/2025 12:51

bon ap

👍 1 🗨

Akshay 02/12/2025 12:53

bon app

3 décembre 2025

Kassad 03/12/2025 12:00

ça avance ?

👍 5 🗨

Ahmed 03/12/2025 12:01

Je recommence le memoire, afin d'avoir une bonne structure selon le sommaire avec les noms de

@Kassad ça avance ?

faycal95100 03/12/2025 12:18

Oui cv je rajoute les dernières captures d'écran de mon code et je peaufine l'ensemble (modifié)

Kassad 03/12/2025 17:21

oki

5 décembre 2025

Akshay 05/12/2025 11:09

Bonjour à tous

Expenses indiqués par Mario, merci de bien vouloir nous remettre vos rapports et présentations Powerpoint pour la certification RNEP2025 DEVEL OPPELUS WEB FULL STACK Niveau 5 (BAC +2) d'ici le 10 décembre 2025.

Pour votre information, vos dossiers doivent être déposés par vos soins sur la plateforme 300k (porteur certificateur) au plus tard 4 semaines avant la date d'examen.

Nous remercions vous vous plus précieusement dès que la date de jury final par le centre certificateur nous sera communiquée.

Bien Cordialement,

👍 2 🗨

Kassad 05/12/2025 14:28

ça avance

faycal95100 05/12/2025 14:29

oui je rajoute des shema uml et paufine la partie 1

@Kassad ça avance

Mathis 05/12/2025 14:32

oui ça avance, on complète avec les screens & schémas

Excalidraw

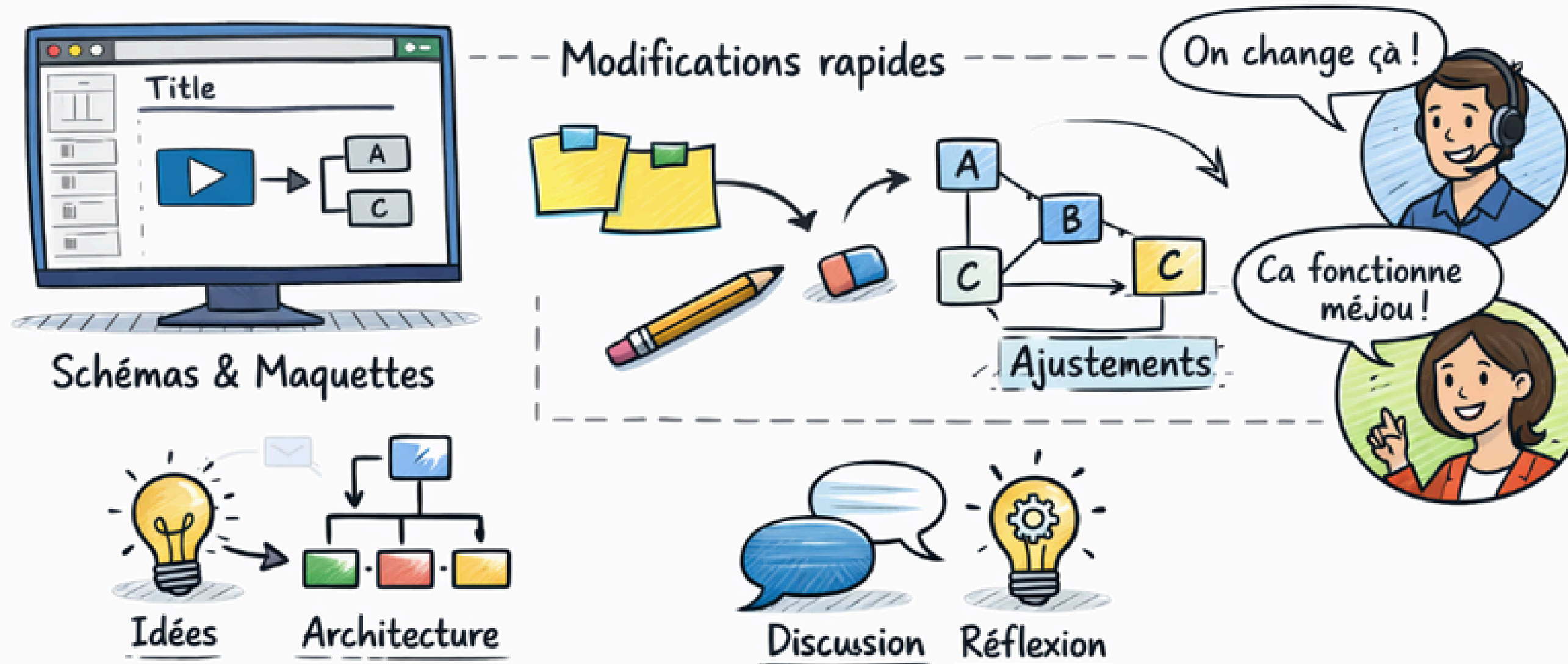
Excalidraw est un outil en ligne qui a permis de créer des schémas et des maquettes simples pour le projet.

Il a facilité la visualisation de l'architecture, l'organisation des idées et la compréhension globale du fonctionnement avant le développement.

Excalidraw a également permis de modifier et d'ajuster rapidement les schémas, rendant les échanges plus clairs et plus efficaces lors des phases de réflexion.

Excalidraw

Outil de création de schémas

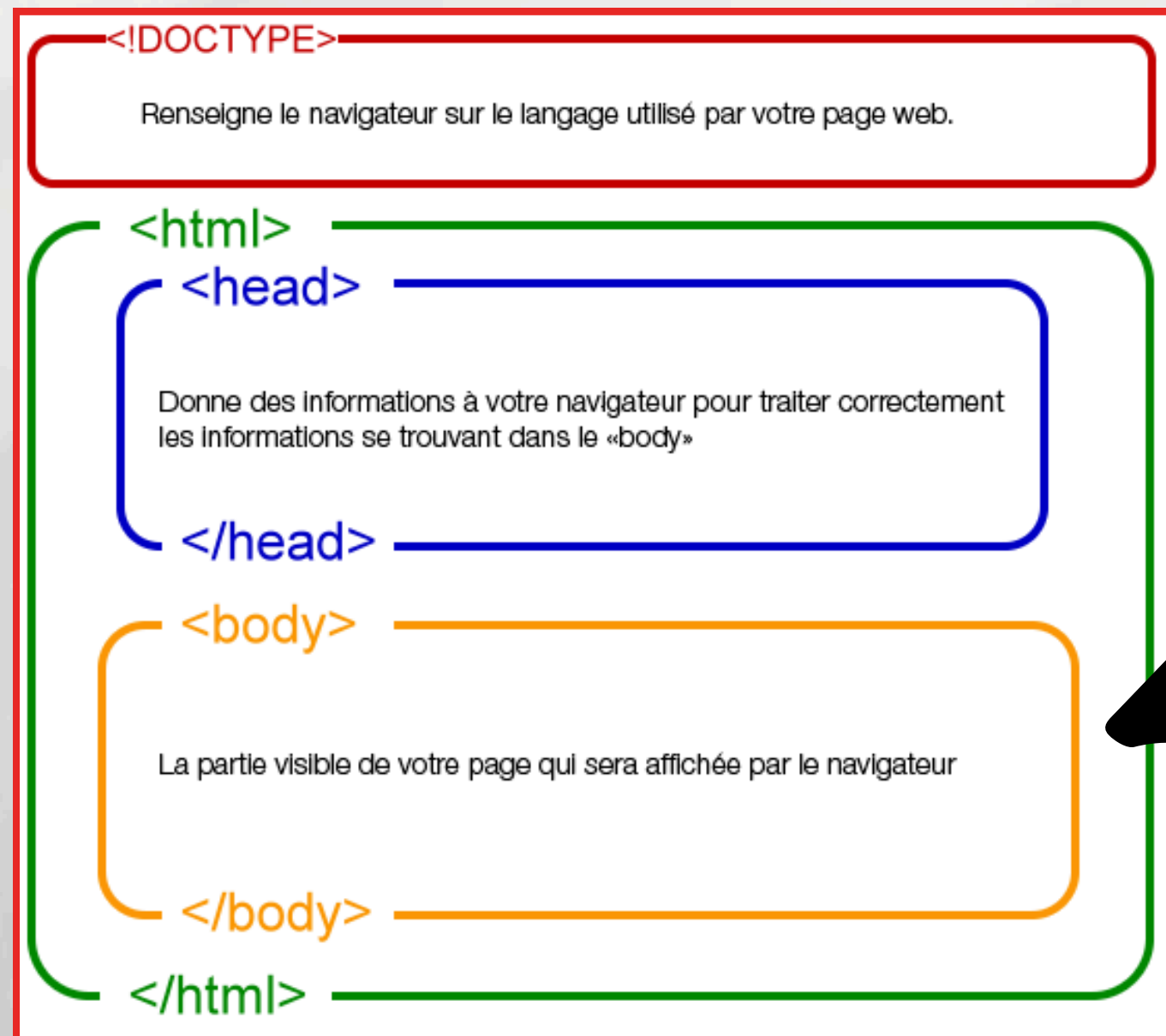


Ce schéma illustre l'utilisation d'**Excalidraw** pour concevoir des schémas et des maquettes de projet.

Il met en avant la visualisation des idées, l'architecture du projet ainsi que la possibilité d'effectuer des ajustements rapides afin d'améliorer la compréhension et la collaboration.

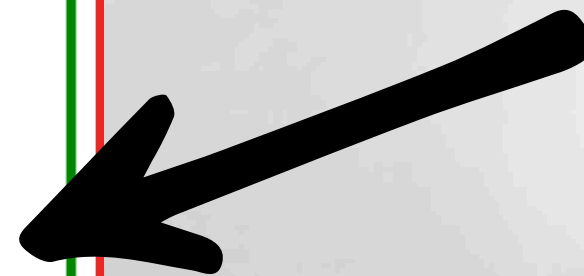
HTML5

HyperText Markup Language



C'est ici que je compte mettre mes balises html afin de structurer ma page comme je le souhaite

<header>, <nav>, <main>, et <footer>



HTML5 & la sémantique par le code

```
<header id="headerOne">
  <nav id="navOne">
    <div class="nav-left" v-if="!userStore.isAuthenticated"> ...
    </div>

    <div class="nav-left" v-if="userStore.isAuthenticated"> ...
    </div>

    <ul class="menu" v-if="!userStore.isAuthenticated"> ...
    </ul>

    <ul class="menu" v-if="userStore.isAuthenticated"> ...
    </ul>

    <div class="nav-right" v-if="!userStore.isAuthenticated"> ...
    </div>

    <div class="nav-right" v-if="userStore.isAuthenticated">
      <div class="user-info">
        <a href="/favoris" class="heart-link" title="Mes favoris">♥</a>

        <div class="user-dropdown"> ...
        </div>
      </div>
    </div>
  </nav>
</header>
```

CSS3

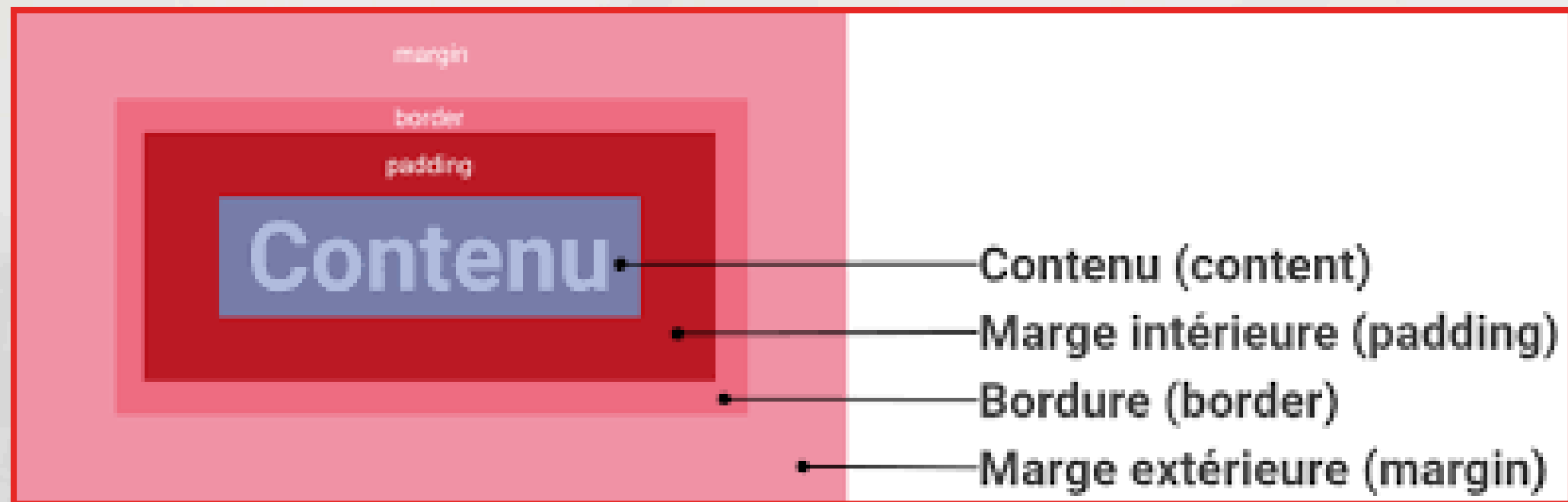
Cascading Style Sheets

J'ai utilisé le langage **CSS3** pour mettre en forme et styliser l'intégralité du site **Portal_job**. Le **CSS** est essentiel pour séparer la présentation (**couleurs, polices, mise en page, etc.**) de la structure **HTML**, assurant ainsi une maintenance plus facile et un code plus clair.

Un aspect crucial de mon travail a été l'adoption d'un design adaptatif (**Responsive Design**). J'ai implémenté les **Media Queries** afin que l'affichage du site s'ajuste parfaitement à toutes les tailles d'écran (ordinateurs, tablettes et mobiles), garantissant une expérience utilisateur optimale sur tous les supports.

CSS3

Cascading Style Sheets



```
<style scoped>

.logo img {
  width: 250px;
}

/* HEADER */
#headerOne {
  background-color: var(--bg);
  box-shadow: 0 6px 16px var(--shadow);
  border-bottom: 1px solid #ccc;
  position: sticky;
  top: 0;
  z-index: 1000;
}

#navOne {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 3rem;
  background-color: #rgb(240, 240, 240);
}

/* LOGO */
.logo img {
  width: 200px;
}

/* MENU PRINCIPAL */
.menu {
  list-style: none;
  display: flex;
  gap: 2.5rem;
  margin: 0;
  padding: 0;
}

</style>
```


Conception Réactive (Responsive Design)

L'un des objectifs clés était d'améliorer l'expérience utilisateur et de moderniser le site. Cela passe nécessairement par le **Responsive Design** (design adaptatif).

Principe :

Le site s'adapte parfaitement à toutes les tailles d'écran (mobiles, tablettes, ordinateurs)

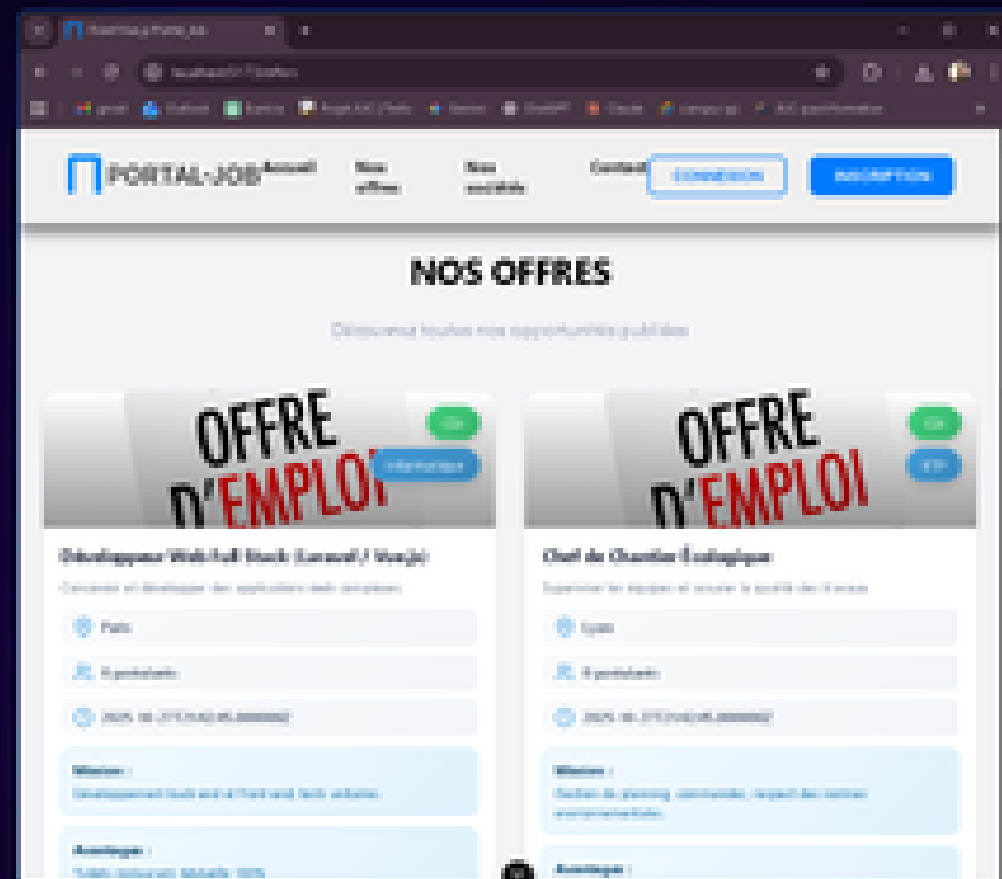
Mise en œuvre :

J'ai utilisé le CSS3 natif et implémenté les Media Queries pour garantir une interface conviviale et intuitive pour tous les utilisateurs

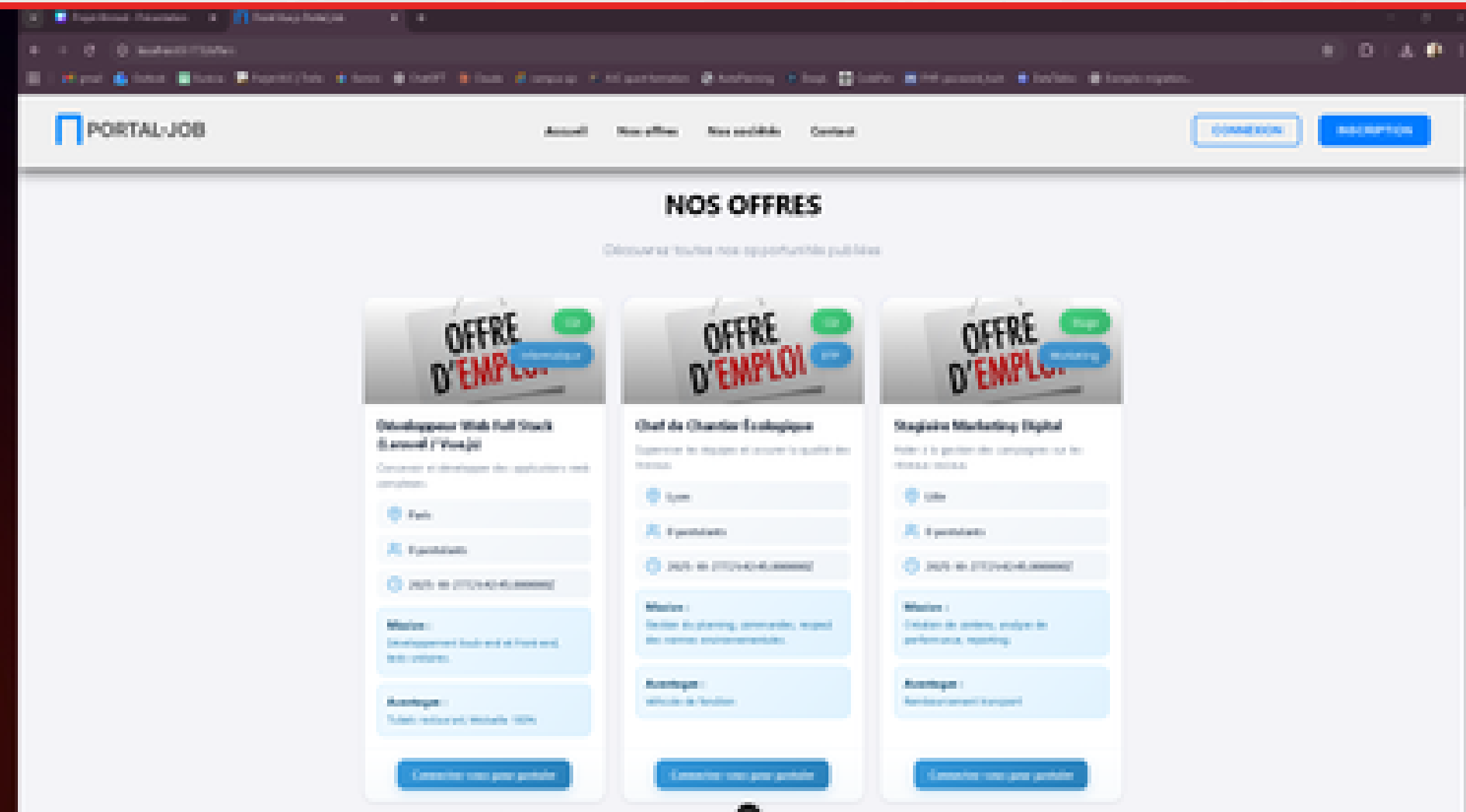
Bénéfice :

Cette approche assure une expérience utilisateur optimale sur tous les supports, ce qui est crucial pour un site d'offres d'emploi.

De ce côté, le site
sous une fenetre retrecie



De l'autre, le site plein écran



GitHub

GitHub c'est une plateforme basée sur le cloud où nous pouvons stocker, partager et travailler avec d'autres pour écrire du code.

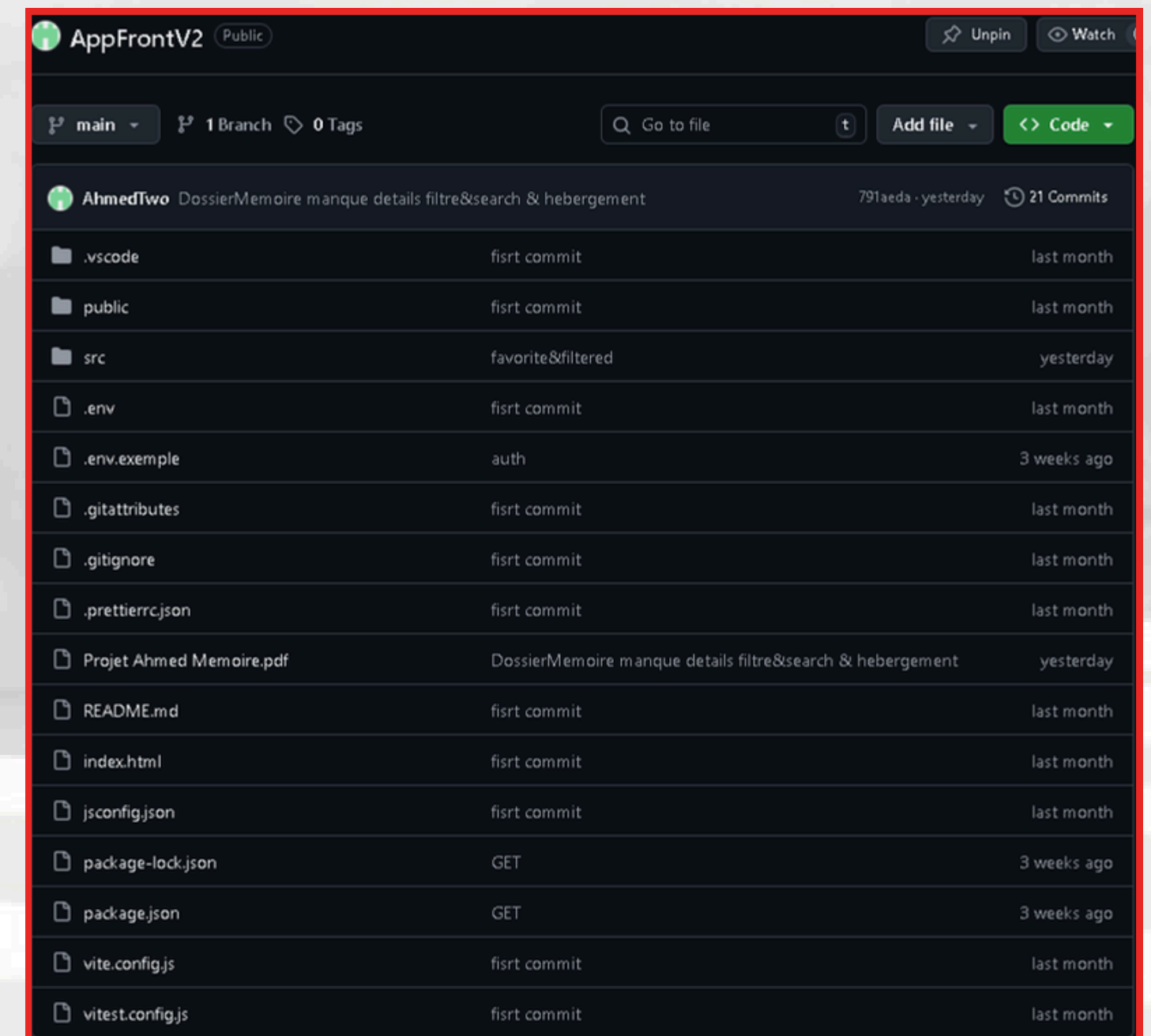
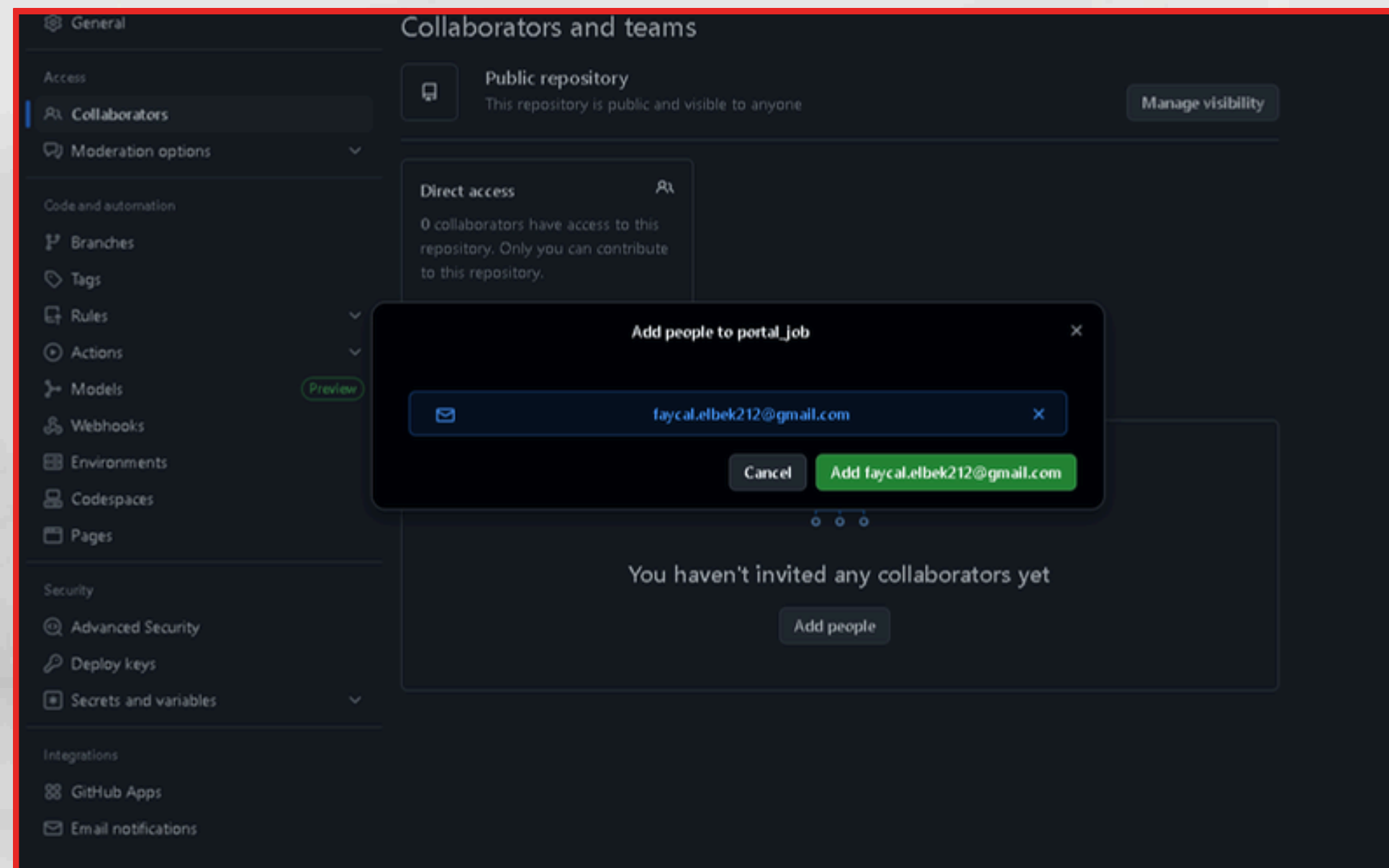
Présenter ou partager votre travail.

Suivre et gérer les modifications apportées à votre code au fil du temps.

```
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/AhmedTwo/portal_job.git
git push -u origin main # ou master selon le nom de ta branche
```

```
C:\wamp64\www\portal_job>git push -f origin master
Enumerating objects: 155, done.
Counting objects: 100% (155/155), done.
Delta compression using up to 16 threads
Compressing objects: 100% (152/152), done.
Writing objects: 100% (155/155), 1.37 MiB | 4.23 MiB/s, done.
Total 155 (delta 17), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (17/17), done.
To https://github.com/AhmedTwo/portal_job.git
+ e0e7d17...5a9a260 master -> master (forced update)
```

pour forcer le push

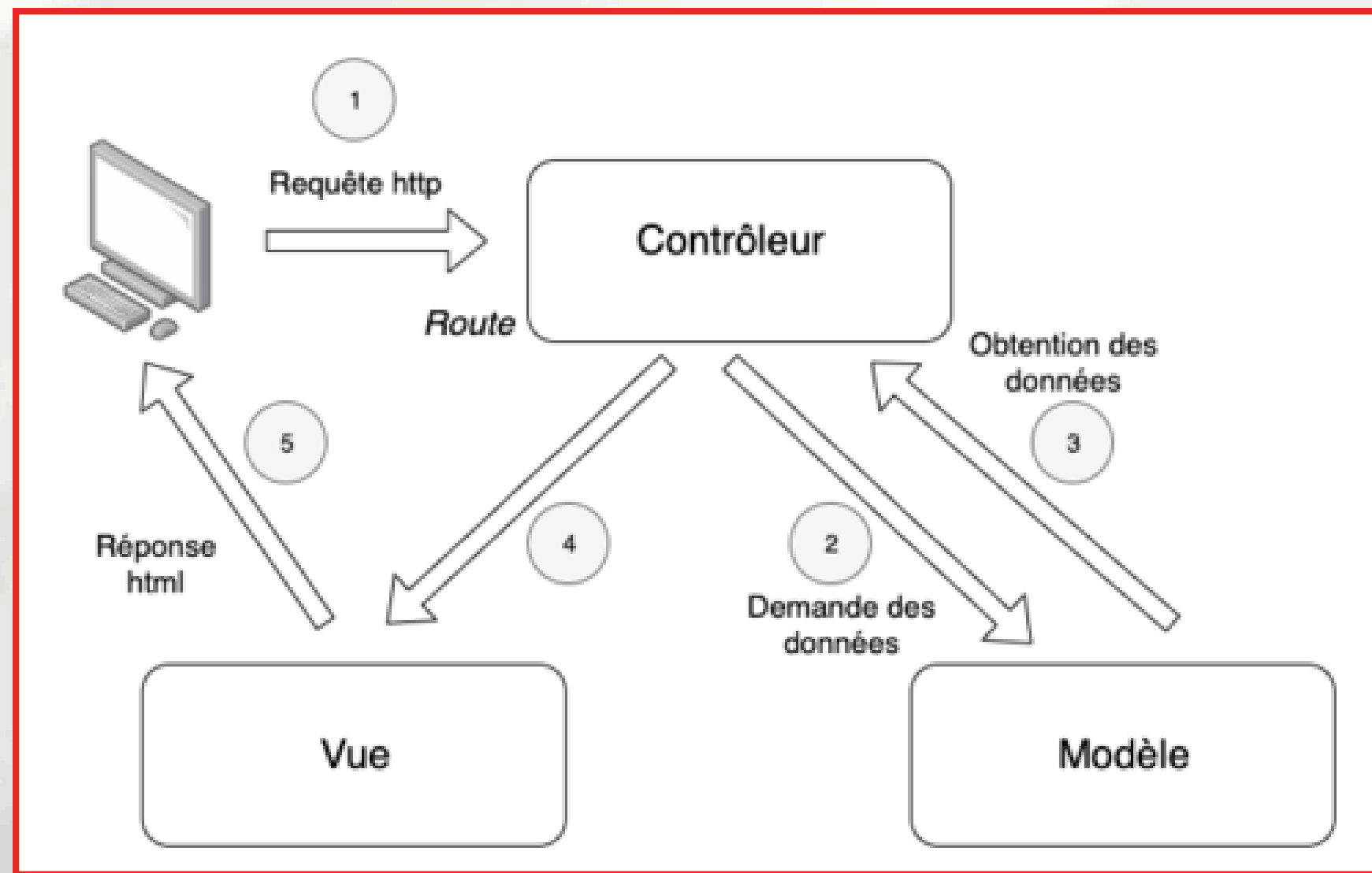


Historique du projet

Du procédurale à l'MVC

Au cours du développement de mon projet, j'ai relevé le défi de le transformer d'une approche procédurale à un modèle basé sur **MVC / programmation orientée objet (POO)** que ça soit pour le **front (Vue.js)** ou le **back (Laravel)**.

Cette transition m'a permis de structurer le code de manière plus modulaire et maintenable. J'ai séparé clairement les responsabilités de l'application en couches distinctes.



1. La Requête (Route) :

L'aiguilleur (reçoit l'appel).

2. Le Contrôleur Orchestre :

Le cerveau (ordonne et décide).

3. Le Modèle Gère les Données :

Le coffre-fort (stocke et calcule).

4. La Vue est Préparée :

Le miroir (affiche le visuel).

5. La Réponse est Servie :

Le résultat (rendu final).

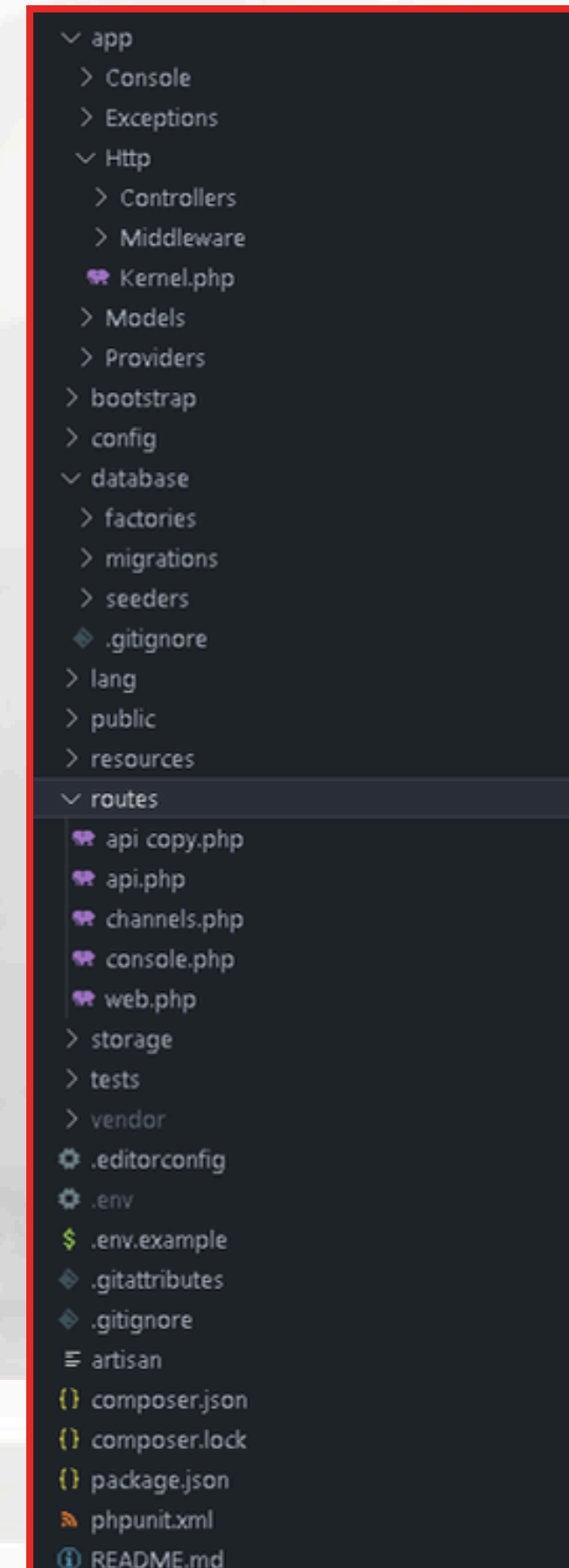
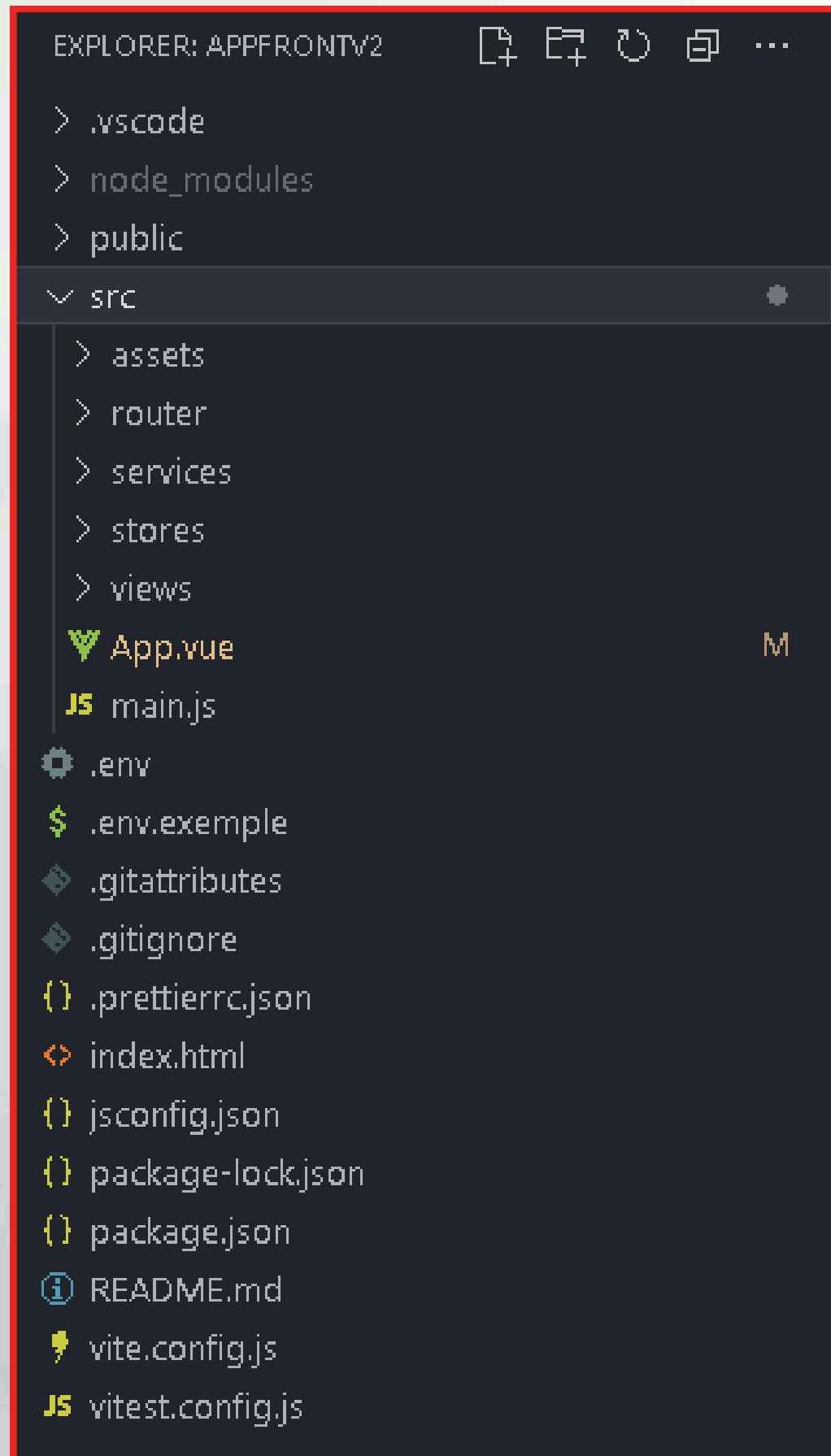
En résumé :

Le Contrôleur demande au Modèle : "**Donne-moi les infos.**"

Le Modèle répond : "**Tiens, voilà les données.**"

Le Contrôleur donne à la Vue : "**Affiche ça proprement.**"

FrontEnd Vue.js

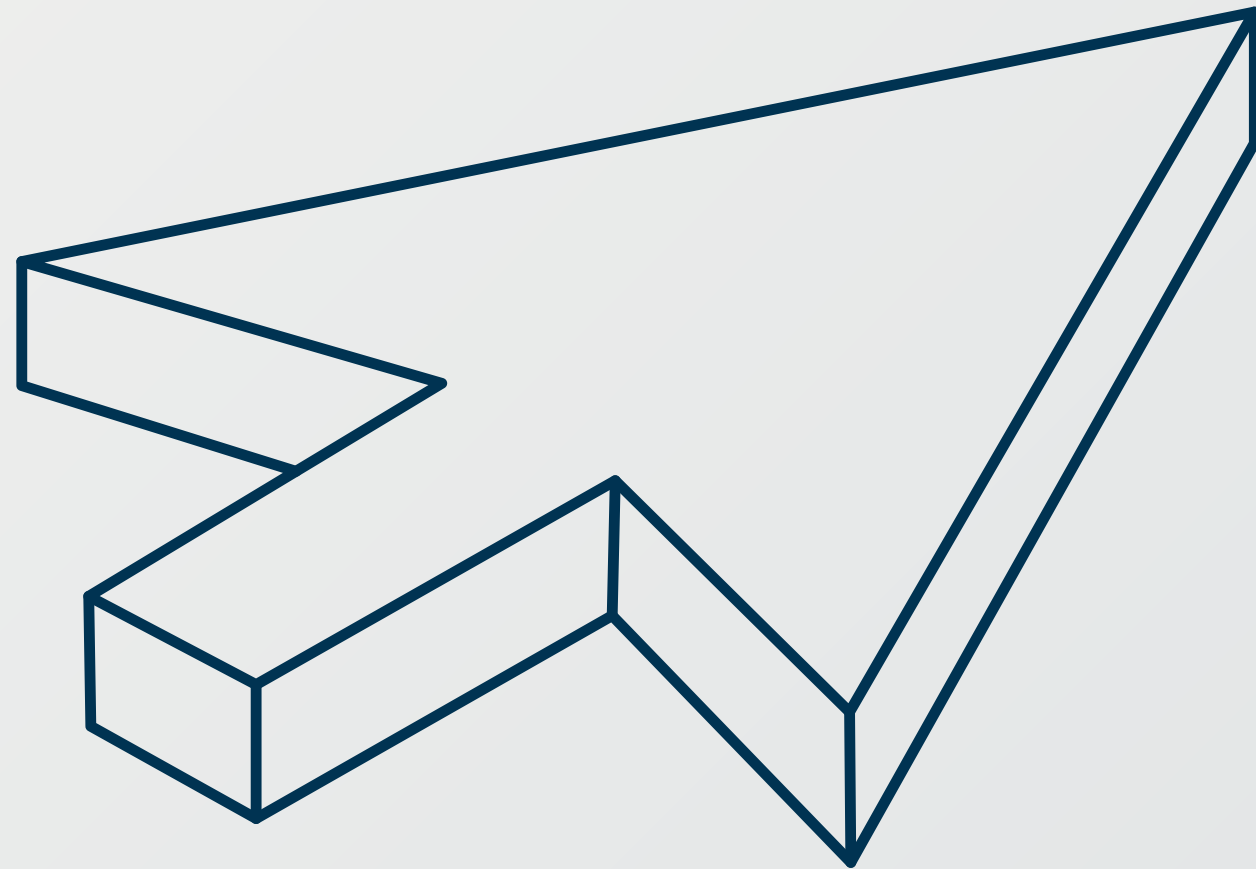


BackEnd Laravel

III -

RNCP 37273 BC03 :

**Développer le Backend d'un site
ou d'une application web / web
mobile**



La base de données

Avant-propos

Nous débuterons par la création d'un dictionnaire de données exhaustif, suivi la schématisation visuelle de la base de données pour définir ses structures et ses relations clés.

Cette approche méthodique garantit une base de données cohérente et efficace pour le système de gestion de [Portal_Job](#).

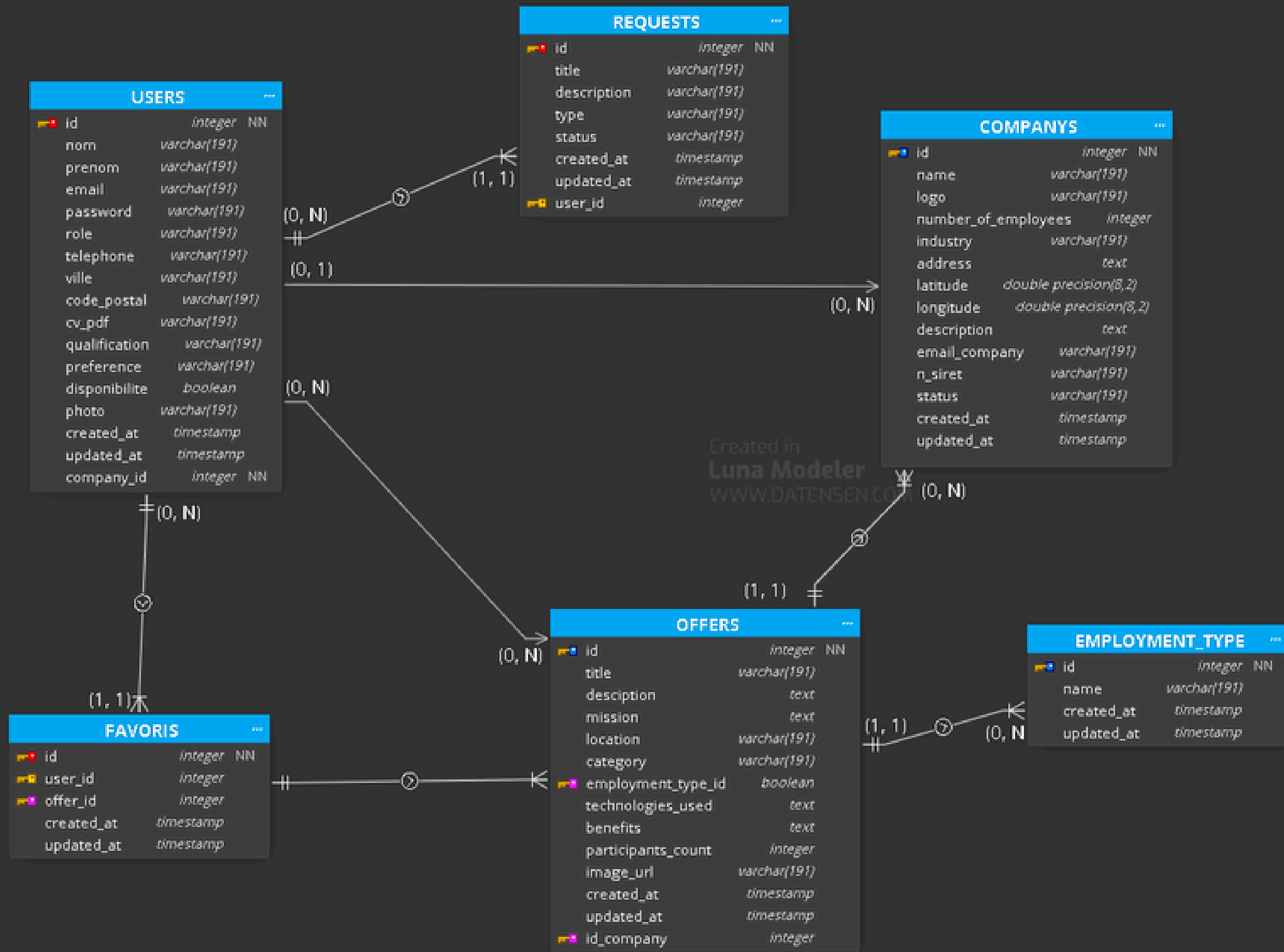
Schématisation de la BDD

MLD Modèle Logique de Données

Pour schématiser visuellement ma base de données et capturer ses relations et structures clés, j'ai utilisé un outil de modélisation de bases de données qui offrent des diagrammes :

J'ai utilisé **LUNA Modeler** (**plan gratuit / linux**)

LUNA Modeler a permis la schématisation visuelle pour définir les structures et les relations clés de la BDD



Cardinalité	Définition courte
0-N	Peut avoir plusieurs, optionnel
1-N	Un lié à plusieurs obligatoirement
0-1	Zéro ou une seule fois
1-0	Relation obligatoire d'un côté
1-1	Un seul avec un seul
N-N	Plusieurs liés à plusieurs
0-0	Aucun lien possible (vide)

PhpMyAdmin



Langue (Language)

Français - French

Connexion

Utilisateur : root

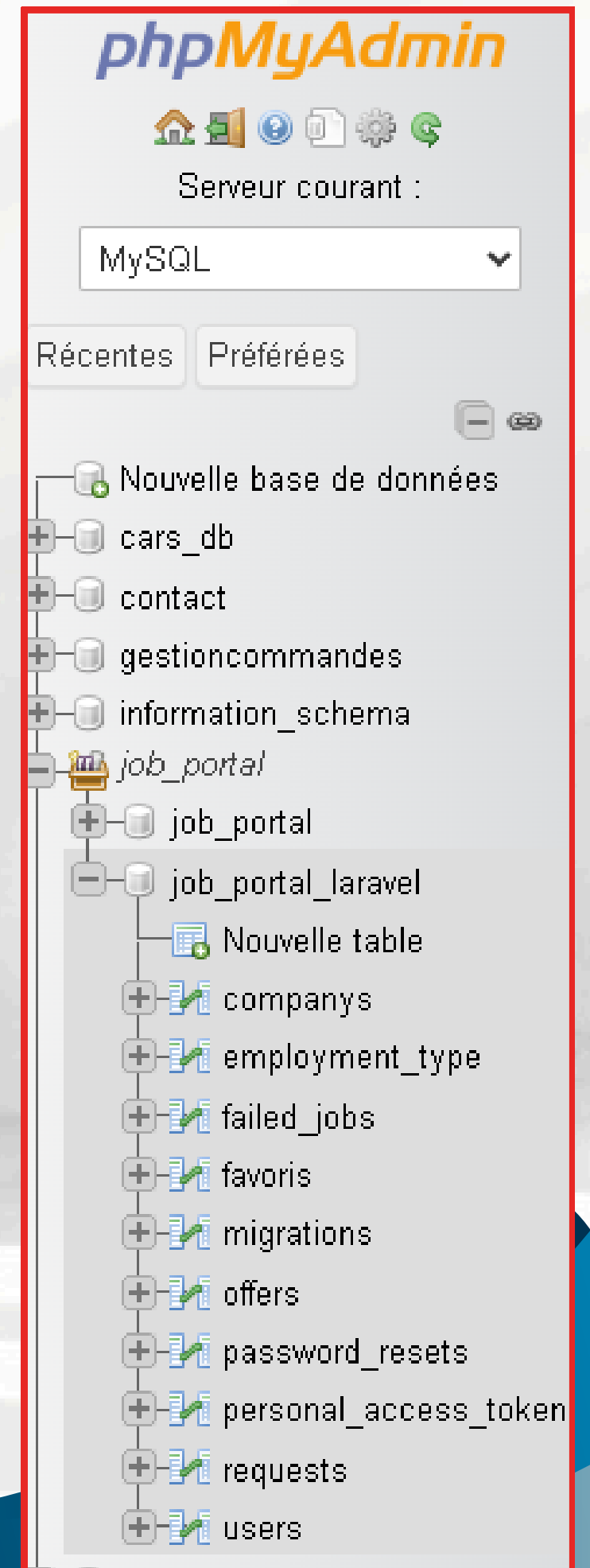
Mot de passe :

Choix du serveur : MySQL

Connexion

PhpMyAdmin est une application web gratuite qui fournit une interface utilisateur graphique pour gérer les bases de données **MySQL** et **MariaDB** via un navigateur.

Il permet aux développeurs et aux administrateurs d'effectuer facilement toutes les opérations de gestion (création, modification, suppression de tables et de données **CRUD**) sans avoir à utiliser de lignes de commande.



Dans une base de données relationnelle, une **table** est l'élément structurel principal, organisant les données en **colonnes** (représentant les attributs ou champs) et en **lignes**. Un **tuple** est simplement le terme technique pour désigner une **ligne unique** dans cette table, représentant un enregistrement complet et spécifique (par exemple, les informations complètes d'un seul employé ou d'une seule offre d'emploi).

Serveur : MySQL3305 > Base de données : job_portal_laravel > Table : users

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
☐ 1	id 🔑	bigint		UNSIGNED	Non	Aucun(e)		AUTO_INCREMENT	Modifier Supprimer Plus
☐ 2	nom	varchar(191)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐ 3	prenom	varchar(191)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐ 4	email 🔑	varchar(191)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐ 5	password	varchar(191)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐ 6	role	varchar(191)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐ 7	telephone	varchar(191)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐ 8	ville	varchar(191)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐ 9	code_postal	varchar(191)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐ 10	cv_pdf	varchar(191)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐ 11	qualification	varchar(191)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐ 12	preference	varchar(191)	utf8mb4_unicode_ci		Oui	NULL			Modifier Supprimer Plus
☐ 13	disponibilite	tinyint(1)			Non	0			Modifier Supprimer Plus
☐ 14	photo	varchar(191)	utf8mb4_unicode_ci		Non	/public/assets/images/userDefault.jpeg			Modifier Supprimer Plus
☐ 15	created_at	timestamp			Oui	NULL			Modifier Supprimer Plus
☐ 16	updated_at	timestamp			Oui	NULL			Modifier Supprimer Plus
☐ 17	company_id 🔑	bigint		UNSIGNED	Oui	NULL			Modifier Supprimer Plus

La connexion à la base de données

Au lieu d'utiliser une fonction `getPDO()` dans un fichier `database.php`,

la connexion à la base de données est gérée nativement par **Laravel**.

Le framework utilise les **variables d'environnement** définies dans le fichier `.env` (**DB_CONNECTION**, **DB_HOST**, etc.) pour établir automatiquement la connexion sécurisée via **Eloquent** (l'**ORM** de **Laravel**).

Fichier : `.env` contient

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=job_portal_laravel
DB_USERNAME=root
DB_PASSWORD=
```


Développement du CRUD

Avant-propos

Explorons la mise en place d'un système **CRUD** (**Create, Read, Update, Delete**) en utilisant la **Programmation Orientée Objet (POO)**.

Ce système permet de gérer les opérations de base sur les données tout en assurant un code clair, réutilisable et maintenable.

Pour illustrer cela, nous utiliserons un modèle de classe représentant les entités clés de notre base de données, chaque classe possédant les méthodes nécessaires à chaque opération **CRUD**.

A présent, présentons les opérations de **CRUD** via le modèle **MVC**, pour chacune de ses méthodes, une explication est fournie.

(CRUD) CREATE

Les opérations de Create (offers) :

La méthode **CREATE** est la première étape du **CRUD**, permettant d'insérer de nouvelles données (**un nouveau tuple**) dans la base de données (par exemple, ajouter une nouvelle offre d'emploi). Elle prend en entrée les informations fournies par l'utilisateur et les prépare de manière sécurisée avant de les enregistrer dans la table correspondante.

```
public function addoffer(Request $requestParam)
{
    $validatedData = $requestParam->validate([
        'title'          => 'required|string|max:255',
        'description'     => 'required|string|max:2000',
        'mission'         => 'required|string|max:255',
        'location'        => 'required|string|max:255',
        'category'        => 'required|string|max:255',
        'employment_type_id' => 'required|integer|exists:employment_type,id',
        'technologies_used' => 'required|string|max:255',
        'benefits'        => 'required|string|max:255',
        'image_url'       => 'required|file|mimes:jpeg,png,jpg,webp|max:2048',
        'id_company'      => 'required|integer|exists:companys,id',
    ], [ ...
    ]));

    try {
        $validatedData['image_url'] = $requestParam->file('image_url')->store('photo_offer', 'public');

        $offer = Offer::create([
            'title'          => $validatedData['title'],
            'description'     => $validatedData['description'],
            'mission'        => $validatedData['mission'],
            'location'       => $validatedData['location'],
            'category'       => $validatedData['category'],
            'employment_type_id' => $validatedData['employment_type_id'],
            'technologies_used' => $validatedData['technologies_used'],
            'benefits'       => $validatedData['benefits'],
            'image_url'      => $validatedData['image_url'],
            'id_company'     => $validatedData['id_company'],
        ]);

        return response()->json([
            'success' => true,
            'message' => 'Offre créée avec succès.',
            'data' => $offer,
        ], 201);
    } catch (\Exception $e) {
        return response()->json([
            'message' => 'Échec de l\'ajout de l\'offre.',
            'error' => $e->getMessage(),
        ], 500);
    }
}
```

(CRUD) READ

Les opérations de Read (offers) :

La méthode **READ** est essentielle pour récupérer les données existantes stockées dans la base de données afin de les afficher à l'utilisateur (**par exemple, lister toutes les offres d'emploi**). Elle peut être utilisée pour obtenir tous les enregistrements ou pour filtrer et afficher un seul tuple spécifique sur la base d'un identifiant ou d'un critère de recherche.

```
public function getOffer()
{
    $data = Offer::select(
        'id',
        'title',
        'description',
        'mission',
        'location',
        'category',
        'employment_type_id', // dans le front on a donc appelé {{ offer.employment_type.name }}
        'technologies_used',
        'benefits',
        'participants_count',
        'image_url',
        'created_at',
        'updated_at',
        'id_company',
    )->with('employment_type')->get();

    return response()->json([
        'success' => true,
        'data' => $data
    ], 200); // code reponse 200 pour success
}
```

(CRUD) UPDATE

Les opérations de Update (offers) :

La méthode **UPDATE** permet de modifier les données existantes dans un enregistrement spécifique de la base de données (**par exemple, changer la description d'une offre d'emploi déjà publiée**).

Pour garantir la cohérence, elle nécessite généralement l'identifiant de l'enregistrement à cibler, ainsi que les nouvelles valeurs à appliquer aux champs.

```
public function updateOffer(Request $requestParam, $id)
{
    $offer = Offer::find($id);

    if (!$offer) {
        return response()->json([
            'success' => false,
            'message' => 'Offre non trouvée',
        ], 404);
    }

    $validatedData = $requestParam->validate([
        'title' => 'sometimes|string|max:255',
        'description' => 'sometimes|string|max:2000',
        'mission' => 'sometimes|string|max:255',
        'location' => 'sometimes|string|max:255',
        'category' => 'sometimes|string|max:255',
        // Ajouter 'integer' et 'exists' pour la clé étrangère afin de faire le lien avec la table employment_type
        'employment_type_id' => 'sometimes|integer|exists:employment_type,id',
        'technologies_used' => 'sometimes|string|max:255',
        'benefits' => 'sometimes|string|max:255',
        'image_url' => 'sometimes|file|mimes:jpeg,png,jpg,webp|max:2048',
    ], [ ...
    ]));

    $offer->update($validatedData);

    return response()->json([
        'success' => true,
        'message' => 'Offre mise à jour avec succès',
        'data' => $offer
    ], 200);
}
```

(CRUD) DELETE

Les opérations de Delete (offers) :

La méthode **DELETE** est utilisée pour supprimer définitivement un ou plusieurs enregistrements de la base de données (**par exemple, retirer une offre d'emploi qui n'est plus valide**). C'est l'opération la plus sensible, car elle requiert une identification précise du tuple à l'aide de son identifiant unique pour éviter toute perte de données accidentelle.

```
public function deleteOffer($id)
{
    $offer = Offer::find($id);

    if (!$offer) {
        return response()->json([
            'succes' => false,
            'message' => 'Offre non trouvée, impossible de la supprimer',
        ], 404);
    }

    try {
        $offer->delete();

        return response()->json([
            'success' => true,
            'message' => 'Offre supprimée avec succès.'
        ], 200);
    } catch (\Exception $e) {
        return response()->json([
            'message' => 'Echec de la suppression de l\'offre',
            'error' => $e->getMessage()
        ], 500);
    }
}
```


Postman

Avant-propos

Postman est un outil qui permet de tester facilement une **API Backend**.

Il sert à envoyer des requêtes (**GET, POST, PUT, DELETE**) et à vérifier les réponses du serveur afin de s'assurer que les fonctionnalités fonctionnent correctement avant de les connecter au **Frontend**.

Il aide ainsi à détecter rapidement les erreurs et à garantir la fiabilité de **l'API** tout au long du développement.

```
// tous les rôles non connecté !!!
Route::middleware(['guest'])->group(
    function () {
        Route::post('/login', [AuthController::class, 'login']); // connexion
        Route::post('/addUser', [CompanyController::class, 'addUser']); // inscription

        Route::get('/count', [UserController::class, 'getCount']); // affichage page d'entrée
        Route::get('/allOffer', [OfferController::class, 'getOffer']);
        Route::get('/offerById/{id}', [OfferController::class, 'getOfferById']);

        Route::get('/allCompany', [CompanyController::class, 'getCompany']);
        Route::get('/companyById/{id}', [CompanyController::class, 'getCompanyById']);
        Route::post('/addCompany', [CompanyController::class, 'addCompany']);
    }
);

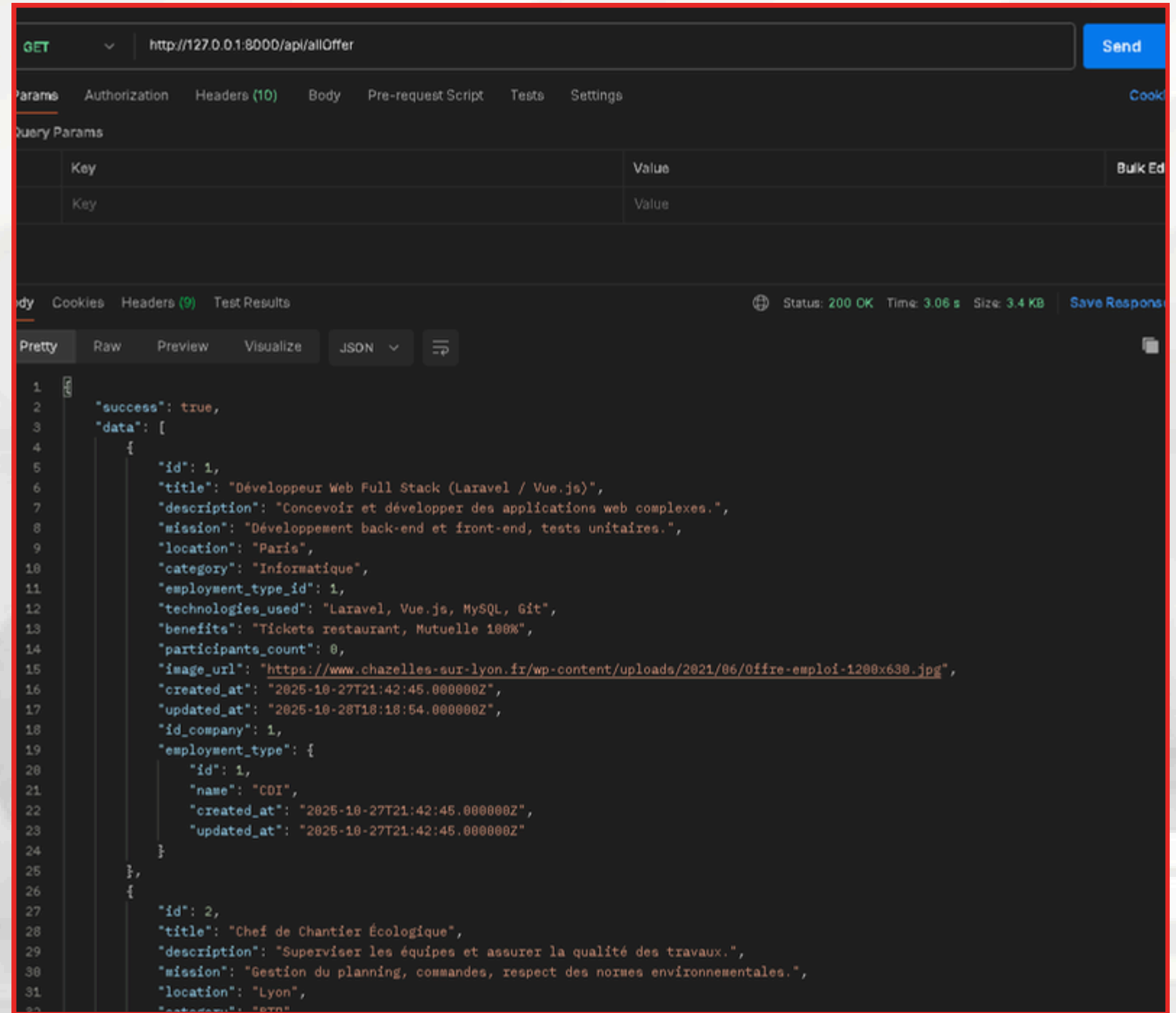
Route::middleware(['auth:sanctum', 'role:admin'])->group(
    function () {
        Route::get('/allUser', [UserController::class, 'getUser']);
        Route::get('/userByRole/{role}', [UserController::class, 'getUserByRole']);
        Route::post('/deleteUser/{id}', [UserController::class, 'deleteUser']);

        Route::get('/allRequest', [RequestController::class, 'getRequest']);
    }
);
```

Flux Postman - API

Postman envoie une Requête **HTTP** (ex: **GET, POST, PUT, DELETE**) vers une **URL** spécifique (**endpoint**) de votre **API**.

L'API Backend reçoit la requête, interagit avec la base de données (**CRUD**), puis renvoie une Réponse **HTTP** contenant le statut (ex: **200 OK**) et les données demandées au client (**Postman**).



Composants BackEnd

Filtrage des Offres

Pour améliorer l'expérience utilisateur du site en ligne, j'ai mis en place un système simple de filtrage des offres d'emplois.

Cette fonctionnalité permet aux utilisateurs de filtrer par catégorie (ex: contrat, domaine...), rendant la navigation et la recherche de produits plus efficaces

Système de filtre

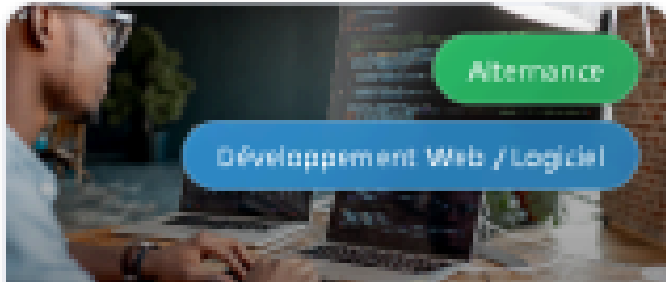
Le **Système de Filtre** permet aux candidats de trouver facilement les offres d'emploi qui les intéressent. Elle améliore significativement l'expérience utilisateur en offrant un **tableau de bord personnalisé** pour gérer et retrouver rapidement les opportunités en fonction de la recherche.

L'interface de recherche est gérée par des champs de saisie **HTML** liés au modèle de données de **Vue.js** :

```
<div class="search-bar-section">
  <div class="search-grid">
    <input
      type="text"
      v-model="searchCity"
      placeholder="Ville (Ex: Paris)"
      class="search-input"
    />
    <input
      type="text"
      v-model="searchContract"
      placeholder="Contrat (Ex: CDI)"
      class="search-input"
    />
    <input
      type="text"
      v-model="searchName"
      placeholder="Nom/Titre (Ex: Développeur)"
      class="search-input"
    />
    <input
      type="text"
      v-model="searchDomain"
      placeholder="Domaine (Ex: Informatique)"
      class="search-input"
    />
  </div>
</div>
```

Systeme de filtre

Recherchez toutes nos opportunités publiques



Alternance

Développement Web / Logiciel

Développeur Full Stack Junior
(Vue.js & Laravel)

Intégration dans une équipe agile pour développer de nouvelles fonctionnalités sur notre plateforme de gestion interne.

12 Rue de la Transformation, 75017 Paris

0 postulants



CDI

Développeur Web Full-Stack

Développeur Web Full-Stack – e-Santé

Développer de nouvelles fonctionnalités pour les applications web BioForme Santé, optimiser les performances et collaborer avec l'équipe médicale pour transformer les besoins métiers en solutions technologiques fiables.

12 Rue des Orchidées, 69007 Lyon

Comme on peut le voir,
Nous entrons dans les champs des informations
qui ensuite nous fera le filtre des offres en
questions.

Si pas de résultats suite aux informations
indiqué dans le(s) champs alors nous allons avoir
un message nous le mentionnant

Composants BackEnd

Favoris des Offres

Pour encore améliorer l'expérience utilisateur du site en ligne, j'ai mis en place un système de favoris des offres d'emplois.

Cette fonctionnalité permet aux utilisateurs de mettre de côté des offres qui les intéressent afin d'y revenir par la suite, sur leur espace 'favoris', rendant la navigation et la recherche d'offres plus efficaces.



Composants BackEnd

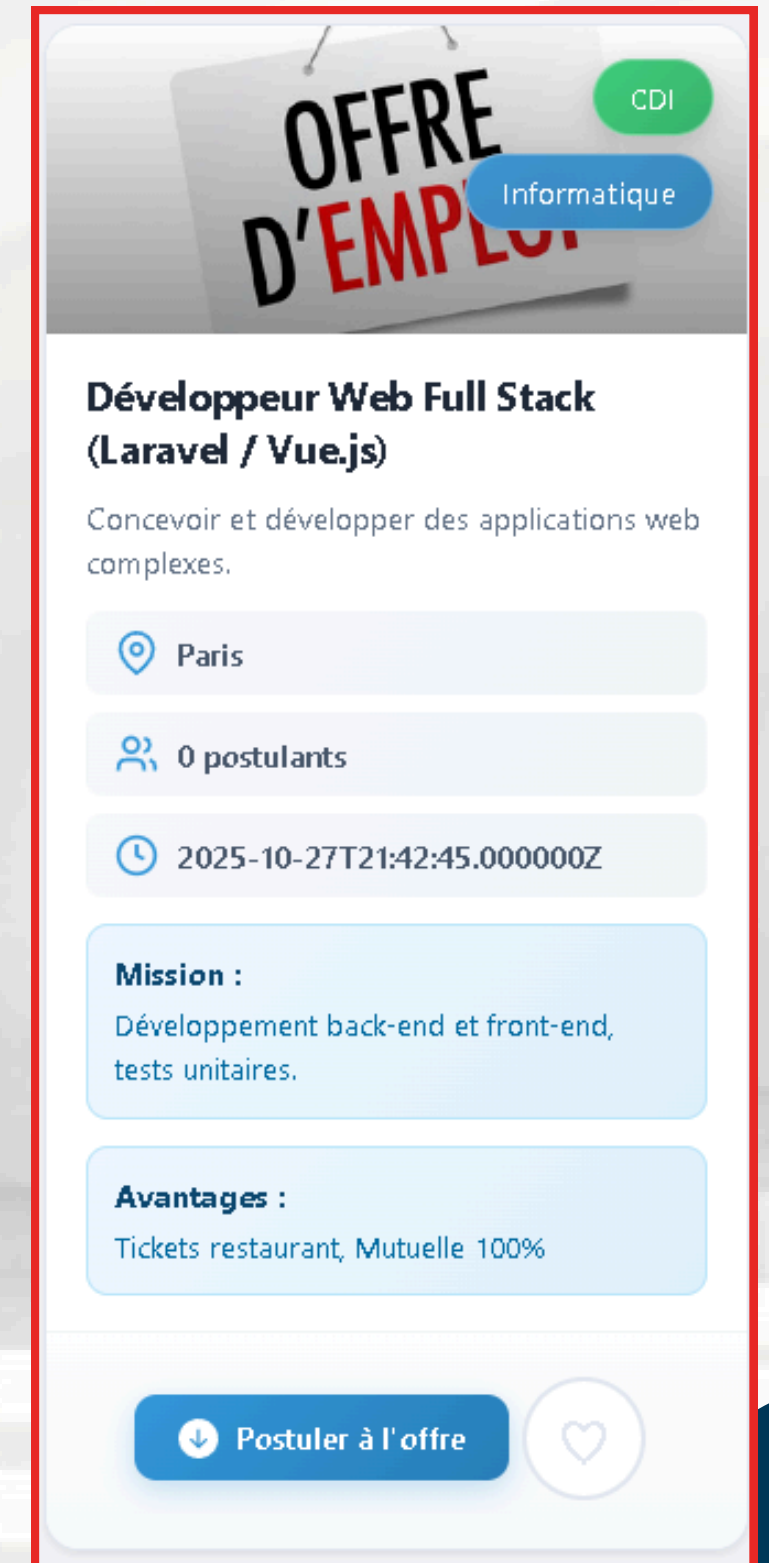
Système de favoris

Le **Système de Favoris** permet aux candidats d'enregistrer et de suivre facilement les offres d'emploi qui les intéressent, sans avoir à postuler immédiatement.

Techniquement, cette fonctionnalité nécessite l'établissement d'une **relation spécifique** entre l'entité Utilisateur (**Candidat**) et l'entité **Offre d'Emploi** dans la base de données. Elle améliore significativement l'expérience utilisateur en offrant un **tableau de bord personnalisé** pour gérer et retrouver rapidement les opportunités.

```
<button
  @click="toggleFavorite(offer.id)"
  class="btn-heart"
  :class="{ 'is-favorite': isFavorite(offer.id) }"
  style="margin: 2%"
  aria-label="Ajouter aux favoris"
>

  <span style="font-size: 35px">
    {{ isFavorite(offer.id) ? '♥' : '♡' }}
  </span>
</button>
```



The image shows a job offer card for a 'Développeur Web Full Stack' position. At the top, there's a header with 'OFFRE D'EMPLOI' and a green 'CDI' badge. Below this, the job title 'Développeur Web Full Stack (Laravel / Vue.js)' is displayed, followed by a description: 'Concevoir et développer des applications web complexes.' The location is 'Paris', and there are '0 postulants'. The date is '2025-10-27T21:42:45.000000Z'. The 'Mission' section describes the role as 'Développement back-end et front-end, tests unitaires.' The 'Avantages' section lists 'Tickets restaurant, Mutuelle 100%'. At the bottom, there's a blue button 'Postuler à l'offre' and a heart icon for favorites.

PhpMailer

Gestion des Emails

J'ai choisi d'intégrer la librairie **PHPMailer** dans mon contrôleur Laravel pour gérer l'envoi des emails de contact, car elle me permet une configuration **SMTP** plus fiable et sécurisée, cruciale pour l'utilisation de services comme Gmail.

Je gère la soumission des données depuis le front-end **Vue.js** qui appelle une **API** vers ce contrôleur, ce qui me garantit une séparation des préoccupations nette entre la présentation et le traitement serveur.

Mon code s'assure de la sécurité en validant les données, en utilisant les variables d'environnement **(.env)** pour les identifiants **SMTP**, et en assurant la robustesse grâce à la gestion des erreurs avec le bloc try-catch.

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=seghiriahmed9@gmail.com
MAIL_PASSWORD= # Le mot de passe de 16 caractères !
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=seghiriahmed9@gmail.com # Doit correspondre à MAIL_USERNAME pour Gmail
MAIL_FROM_NAME="${APP_NAME}"
```



```

class ContactController extends Controller
{
    public function submitContact(Request $request)
    {
        // on valide les des données du form contact.vue
        $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email|max:255',
            'subject' => 'required|string|max:255',
            'message' => 'required|string',
        ]);

        // on recup ensuite les des données validées
        $name = $request->input('name');
        $fromEmail = $request->input('email');
        $subject = $request->input('subject');
        $messageContent = $request->input('message');

        // Configuration de PHPMailer pour l'envoi
        $mail = new PHPMailer(true);

        try {
            // Configuration SMTP
            $mail->isSMTP();
            $mail->Host = env('MAIL_HOST');
            $mail->SMTPAuth = true;
            $mail->Username = env('MAIL_USERNAME');
            $mail->Password = env('MAIL_PASSWORD');
            $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
            $mail->Port = env('MAIL_PORT');
            $mail->CharSet = 'UTF-8';

            // Expéditeur technique : Doit correspondre à l'utilisateur SMTP (pour l'authentification)
            // DOIT être MAIL_USERNAME pour Gmail
            $auth_email = env('MAIL_USERNAME');
            $mail->setFrom($auth_email, 'Formulaire Contact Portal_Job');

            // Destinataire : Votre adresse e-mail personnelle/d'administration
            $mail->addAddress('seghiriahmed9@gmail.com');

            // Adresse de Réponse : L'e-mail de l'utilisateur. C'est ici que vous répondez.
            $mail->addReplyTo($fromEmail, $name);

            // Contenu du message
            $mail->isHTML(false);
            $mail->Subject = "Nouveau message de contact : " . $subject;
            $mail->Body = "De: {$name} ({$fromEmail})\n\nSujet: {$subject}\n\nMessage:\n{$messageContent}";

            $mail->send();

            // Succès
            return response()->json(['message' => 'Votre message a été envoyé avec succès !'], 200);
        } catch (Exception $e) {
            // Échec
            // Loggez l'erreur pour la déboguer
            Log::error("Erreur d'envoi PHPMailer: " . $mail->ErrorInfo);

            return response()->json(['error' => "Erreur lors de l'envoi du mail. Veuillez réessayer."], 500);
        }
    }
}

```

`$request->validate(...)` : Validation immédiate et sécurité des données.

`$mail = new PHPMailer(true);` : Initialisation du moteur d'envoi d'emails.

`$mail->isSMTP();` : Utilisation du protocole SMTP standard.

`$mail->Host = env('MAIL_HOST');` : Configuration des identifiants via variables .env

`$mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;` : Connexion sécurisée TLS chiffrée.

`$mail->addReplyTo($fromEmail, $name);` : Réponse directe à l'utilisateur facilitée.

`$mail->send();` : Exécution de l'envoi de l'email

`try { ... } catch (Exception $e) { ... }` : Gestion robuste des erreurs d'envoi.

Deploiement

Le projet a été migré d'un environnement local vers un **VPS** sous architecture **LAMP**, garantissant une flexibilité totale pour la configuration de **PHP-FPM** et d'**Apache**.

Cette mise en ligne sécurisée via **SSL/TLS** assure la protection des données des utilisateurs tout en offrant une base solide pour le référencement naturel.

Deploiement

Points clés du déploiement :

Infrastructure robuste : Utilisation d'un serveur **Linux** avec optimisation des performances (*MPM Apache, Buffer pools MySQL*).

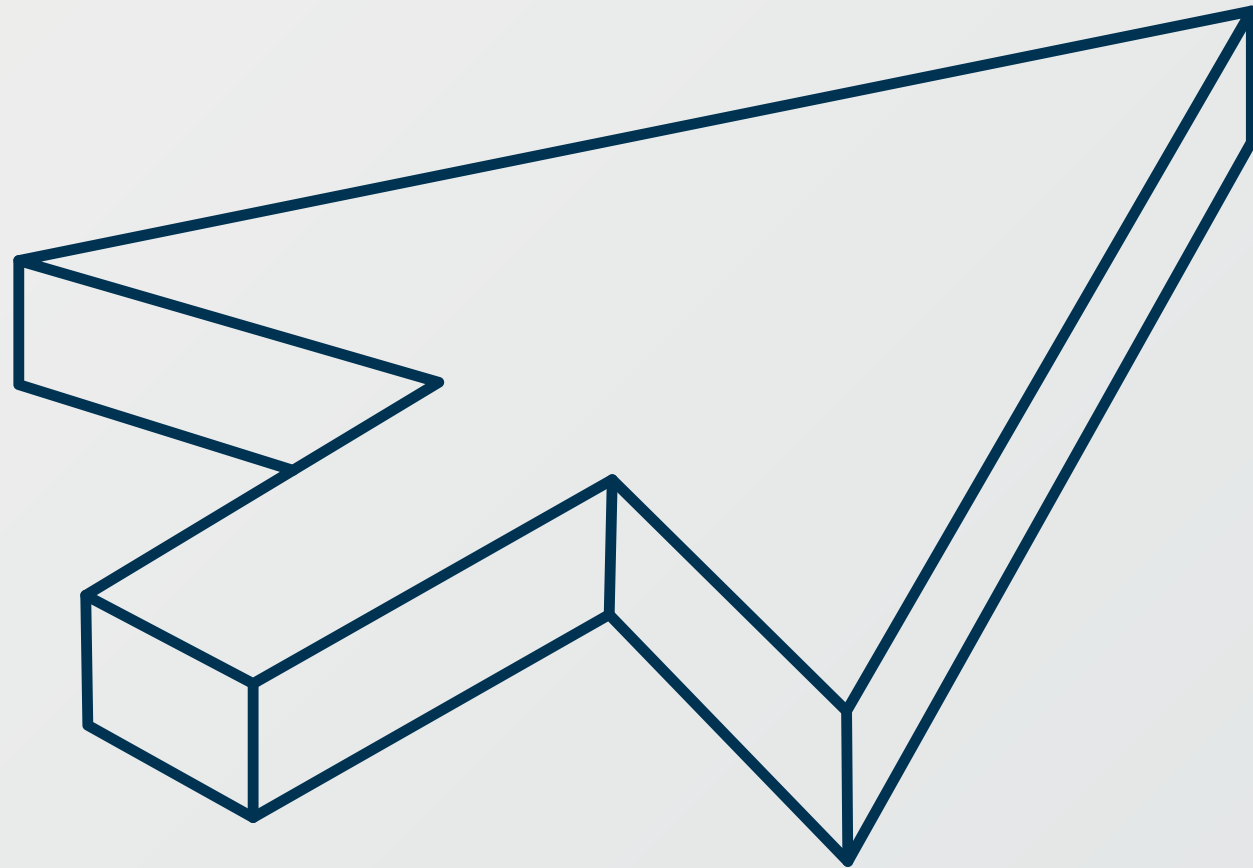
Sécurité renforcée : Mise en place d'un pare-feu iptables et déploiement du protocole **HTTPS**.

Maintenance & SEO : Configuration de **Virtual Hosts** et optimisation du temps de chargement pour un service fluide et pérenne.



IV -

CONCLUSION



Bilan du Projet Portal_Job

Ce projet représente une étape significative dans la maîtrise des technologies full-stack, notamment avec le framework **Laravel** pour le **Backend**. Malgré les défis rencontrés, notamment l'optimisation des requêtes complexes, j'ai réussi à implémenter l'intégralité des **fonctionnalités clés** pour une plateforme d'emploi fonctionnelle ou alors cela sera mis en place d'ici peu.

Le développement de **Portal_job** est la preuve concrète de ma **persévérance**, de ma **capacité à structurer un projet complet** et à appliquer les meilleures pratiques de développement.

Perspectives d'Évolution

Le projet **Portal_job** a un fort potentiel d'évolution.

Les prochaines étapes incluent l'intégration d'un **système d'intelligence artificielle** pour des recommandations d'offres plus pertinentes et l'ajout d'une **messagerie interne** pour faciliter la communication directe entre candidats et recruteurs et admin si besoin.

Conclusion

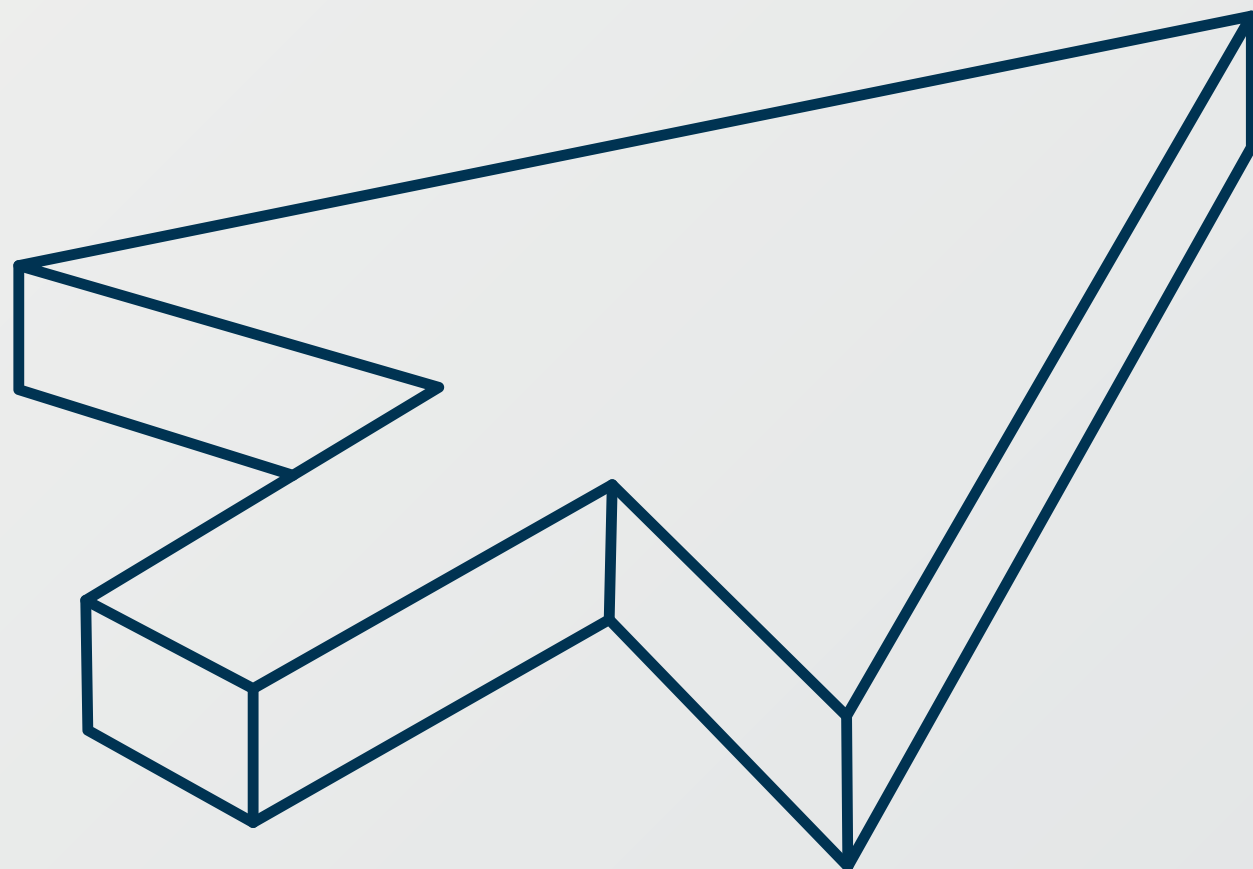
En conclusion ,

La refonte de Portal_job est un succès qui démontre ma capacité à concevoir et développer une plateforme complète, sécurisée et orientée utilisateur, du Frontend au Backend.

Ce projet m'a permis de solidifier mes compétences en Laravel, Vue.js (encore du travail pour ce langage), POO et Gestion de Base de Données et je suis prêt à appliquer cette expertise aux défis futurs du développement web.

V -

ANNEXES



Fichier : api.php (racine du projet back)

```
// Rôles : company, admin
Route::middleware(['auth:sanctum', 'role:company,admin'])->group(
    function () {
        // CORRECTION 1 : Remplacer Route::post par Route::get pour la lecture
        Route::get('/userById/{id}', [UserController::class, 'getUserById']);

        Route::post('/userUpdate/{id}', [UserController::class, 'updateUser']);

        Route::post('/addOffer', [OfferController::class, 'addOffer']);
        Route::post('/offerUpdate/{id}', [OfferController::class, 'updateOffer']);
        Route::delete('/deleteOffer/{id}', [OfferController::class, 'deleteOffer']);

        Route::post('/companyUpdate/{id}', [CompanyController::class, 'updateCompany']);
        Route::delete('/deleteCompany/{id}', [CompanyController::class, 'deleteCompany']);

        Route::get('/requestById/{id}', [RequestController::class, 'getRequestById']);
        Route::post('/requestUpdate/{id}', [RequestController::class, 'updateRequest']);
        Route::delete('/deleteRequest/{id}', [RequestController::class, 'deleteRequest']);
    }
);

// Rôles : candidat, admin
Route::middleware(['auth:sanctum', 'role:candidat,admin'])->group(
    function () {
        // CORRECTION 2 : Remplacer Route::post par Route::get pour la lecture
        Route::get('/userById/{id}', [UserController::class, 'getUserById']);

        Route::post('/userUpdate/{id}', [UserController::class, 'updateUser']);

        Route::get('/requestById/{id}', [RequestController::class, 'getRequestById']);
        Route::post('/requestUpdate/{id}', [RequestController::class, 'updateRequest']);
        Route::delete('/deleteRequest/{id}', [RequestController::class, 'deleteRequest']);

        Route::get('/favorisById/{id}', [FavorisController::class, 'getFavorisById']);
        Route::post('/addFavoris', [FavorisController::class, 'addFavoris']);
        Route::delete('/deleteFavoris/{id}', [FavorisController::class, 'deleteFavoris']);
    }
);
```

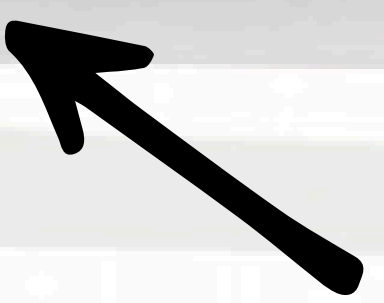
Import de mes controllers qui
eux s'occupent du CRUD et autres
fonctions



```
<?php

use App\Http\Controllers\AuthController;
use App\Http\Controllers\OfferController;
use App\Http\Controllers\FavorisController;
use App\Http\Controllers\RequestController;
use App\Http\Controllers\UserController;
use App\Http\Controllers\CompanyController;
use Illuminate\Support\Facades\Route;
use Illuminate\Http\Request;
```

Les Routes qui s'occupent de la
communication avec la bdd les
controllers et donc dans l'url



Fichier : index.js (projet front)

Import de mes Vue qui eux
s'occupent de faire appel à
mon html css js



```
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'homepage',
      component: HomeView,
    },
    {
      path: '/SignIn',
      name: 'connexion',
      component: Connexion,
    },
    {
      path: '/SignUp/addUser/',
      name: 'inscription',
      component: Inscription,
    },
    {
      path: '/SignIn/passwordForget',
      name: 'motDePasse',
      component: Password,
    },
    {
      path: '/SignIn/applyCompany',
      name: 'ajoutSociete',
      component: ApplyCompany,
    },
    {
      path: '/Home',
      name: 'Accueil',
      component: Accueil,
    },
    {
      path: '/Home/apply',
      name: "Postuler à l'offre",
      component: Apply,
    },
  ],
})
```

history : Définit la manière dont le routeur gère l'historique de navigation dans le navigateur, ici en utilisant le mode HTML5 History (**createWebHistory**).

path : Spécifie le **chemin URL** exact auquel cette route doit correspondre.

name : Fournit un **nom unique** pour cette route, permettant une navigation facile et programmatique sans avoir besoin de **l'URL** complète.

component : Indique la vue **Vue.js** (le composant) qui doit être chargée et rendue lorsque le **path** est activé.

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import Connexion from '../views/SignIn.vue'
import Inscription from '../views/SignUp.vue'
import Password from '../views/PasswordForget.vue'
import ApplyCompany from '../views/ApplyCompany.vue'
import Accueil from '../views/Home.vue'
import Apply from '../views/ApplyOffer.vue'
import Offers from '../views/Offers.vue'
import Companys from '../views/Companys.vue'
import CompanyDetails from '../views/CompanyDetails.vue'
import myRequest from '../views/MyRequest.vue'
import Request from '../views/Request.vue'
import DashboardCompany from '../views/DashboardCompany.vue'
import DashboardAdmin from '../views/DashboardAdmin.vue'
import Favoris from '../views/Favoris.vue'
import Profil from '../views/Profil.vue'
import Contact from '../views/Contact.vue'
import OffersCompany from '../views/OffersCompany.vue'
import UpdateMyRequest from '../views/UpdateMyRequest.vue'
import AddMyRequest from '../views/AddMyRequest.vue'
import UpdateOfferById from '../views/UpdateOffer.vue'
import UpdateCompanyByID from '../views/UpdateCompany.vue'
import UpdateProfil from '../views/UpdateProfil.vue'
```